Perché nasce MapReduce?

L'analisi di grandi quantità di dati (Big Data) non può essere svolta efficacemente da un singolo computer per problemi di efficienza nella memoria, e nell'accesso ad essa.

Architettura distribuita

Invece di un solo computer si utilizza un **cluster** composto da: 100, 1000 nodi; CPU; RAM; dischi locali; reti di collegamento.
Ogni nodo esegue una parte del lavoro.
Classico Divide et Impera.

Distributed File System (DFS)

Per memorizzare enormi quantità di dati viene utilizzato un file system distribuito, le cui caratteristiche sono: file in GB/PB; il file viene diviso in **chunk** 16/64MB; ogni chunk viene replicato; le copie sono conservate su macchine differenti.

Componenti del DFS

Master (NameNode)

Conosce:

- dove si trovano i file;
- dove sono i chunk;
- quali repliche esistono.

Non contiene i dati.

Chunk Server

Contiene realmente i dati.
Ogni server:

- memorizza alcuni chunk;
- esegue anche il calcolo.

Questa è la filosofia di MapReduce:
Bring computation to the data
cioè: portare il codice dove si trovano i dati.

Cos'è MapReduce?

È un modello di programmazione distribuita.
Il programmatore deve scrivere solamente due funzioni:

- **Map**
- **Reduce**

Tutto il resto viene gestito automaticamente:

- parallelizzazione;
- comunicazione;
- sincronizzazione;
- gestione errori;
- scheduling.

Le tre fasi di MapReduce

1. **Map**

Ogni nodo legge un pezzo del file. Per ogni elemento produce una o più coppie: **(key, value)**
Le chiavi possono ripetersi.

2. **Shuffle & Sort**

Il sistema:

- raccoglie tutte le coppie;
- raggruppa quelle con la stessa chiave;
- ordina le chiavi;
- invia ogni gruppo allo stesso reducer.

Questa fase è completamente automatica.

3. **Reduce**

Per ogni chiave: **(key, [v1,v2,v3,...])** la funzione Reduce combina tutti i valori. Produce infine il risultato definitivo.

Esecuzione

Il sistema crea:

- un **Master**
- molti **Worker**

Esistono:

- Map Worker
- Reduce Worker

Il Master:

- assegna i task;
- controlla lo stato;
- redistribuisce il lavoro;
- rileva i guasti.

Se fallisce un Map Worker

Si rieseguono tutte le Map svolte da quella macchina, perché gli output intermedi erano salvati localmente.

Se fallisce un Reduce Worker

È più semplice: il task viene semplicemente rilanciato su un'altra macchina. Il Master gestisce automaticamente tutto questo.

Numero di Map e Reduce

Map

Devono essere molte.
Di solito: 1 Map per ogni chunk

Reduce

Generalmente sono meno. Troppi reducer producono overhead. Troppo pochi riducono il parallelismo.

Miglioramenti (Refinements)

Backup Tasks

Alcune macchine sono lente ("stragglers"). Verso la fine del job viene lanciata una copia dello stesso task. Vince quella che termina prima.

Combiners

Sono una mini-Reduce locale.
Invece di inviare {(the,1), (the,1), (the,1)} si invia {(the,3)}. Può essere usato solo se l'operazione è associativa e commutativa.

Partition Function

Decide quale reducer riceverà una certa chiave.
Default: hash(key) mod R

Ma può essere personalizzata. Ad esempio tutte le pagine dello stesso sito possono andare allo stesso reducer.

Quando NON usare MapReduce

Non è adatto a dati modificati frequentemente.

Join con MapReduce

Per eseguire il join: R(A,B); S(B,C)
Map: R(a,b) -> (b,(a,R)); S(b,c) -> (b,(c,S))
Lo Shuffle porta tutte le tuple con la stessa chiave **B** allo stesso reducer.
Reduce: combina le tuple producendo: (a,b,c)

Costi

Si valutano principalmente due tipi di costo.

Communication Cost

Quantità totale di dati trasferiti tra i nodi. È generalmente il costo dominante.

Computation Cost

Tempo effettivo di calcolo della CPU. Nella maggior parte dei casi è inferiore al costo di comunicazione, perché spostare grandi quantità di dati sulla rete o dal disco alla memoria è più lento dell'elaborazione stessa.

Dipendenze Funzionali Rilassate (RFD)

Le **Relaxed Functional Dependencies** mantengono il concetto di vincolo, ma introducono maggiore flessibilità.

I due criteri di rilassamento

Le RFD si basano su due strategie principali.

Rilassamento sul confronto (Attribute Comparison) A ≈ B

Si possono usare:

- Edit Distance
- Jaro Distance
- Similarity Functions
- Matching Operators
- Metriche di distanza

L'obiettivo è confrontare valori semanticamente equivalenti anche se scritti in modo diverso.

Rilassamento sull'Extent

Una dipendenza non deve necessariamente essere valida per tutte le tuple.
Può valere:

- per la maggior parte delle tuple;
- oppure soltanto per un sottoinsieme.

Coverage Measure

Per misurare quanto una RFD è soddisfatta si usa una **Coverage Measure**. Essa misura il grado di soddisfacimento della dipendenza. Basata su:

- confidenza
- probabilità
- impurità
- g3 error
- cardinalità del dominio.

Confidenza

La confidenza indica la percentuale (o il numero) di tuple che rispettano la dipendenza. Maggiore è la confidenza, più affidabile è la dipendenza.

Condizioni

In alternativa alla confidenza, una dipendenza può essere limitata mediante condizioni. Solo le tuple che soddisfano tale condizione devono rispettare la dipendenza.

Definizione generale di RFD

Una RFD è composta da quattro elementi fondamentali:

1. **Dominio di applicazione**
2. **Vincoli sul lato sinistro (LHS)**
3. **Vincoli sul lato destro (RHS)**
4. **Misura di copertura (Coverage Measure)**

In pratica una RFD afferma: Se due tuple soddisfano i vincoli sul lato sinistro, allora quasi sempre devono soddisfare quelli sul lato destro.

Classificazione delle RFD

Dipendenze che rilassano sull'Extent

AFD (Approximate Functional Dependency)

Una FD valida per **quasi tutte** le tuple.
Misura principale: g3 error

PuD (Purity Dependency)

Utilizza il concetto di **impurità**.
L'impurità misura quanto una partizione dei dati è coerente con un'altra.
Impurità bassa → dipendenza forte.

NuD (Numerical Dependency)

Impone un limite alla cardinalità.
Esempio: Ogni reparto → al massimo 10 stanze

Dipendenze probabilistiche

Tre varianti: PD, soft FD, pFD.
Tutte sostituiscono la validità assoluta con una probabilità.

CD (Constrained Dependency)

La dipendenza vale solo per tuple che soddisfano un determinato vincolo.

CFD (Conditional Functional Dependency)

Molto simile alla CD, ma usa un **Pattern Tableau**, cioè una tabella con: costanti, variabili ("-"); per selezionare le tuple interessate.

Dipendenze che rilassano sul confronto

MFD (Metric Functional Dependency)

Utilizza una distanza metrica.

ND (Neighborhood Dependency)

Usa una **closeness function** che assegna un valore tra 0 e 1 alla similarità. Ogni attributo ha una soglia di accettazione.

SFD (Similarity Functional Dependency)

Si basa sulle **relazioni di tolleranza**. Due valori possono essere considerati equivalenti pur non essendo identici.

TMFD (Type-M Functional Dependency)

Pensata per dati multimediali. Confronta immagini, ECG, audio ecc. mediante funzioni di similarità specifiche per ogni attributo.

MD (Matching Dependency)

Serve a stabilire quando record appartenenti a database differenti rappresentano la stessa entità.

RFD ibride

CoD (Comparable Dependency)

Generalizza molte RFD. Permette di confrontare attributi diversi appartenenti a database differenti. Può usare contemporaneamente: uguaglianza, distanza, matching.

DD (Differential Dependency)

Esprime vincoli sulla differenza tra valori.

- Le **FD classiche** richiedono uguaglianza esatta e validità su tutte le tuple.
- Le **RFD** nascono per gestire dati reali, rumorosi o provenienti da più sorgenti.
- Esistono **due modalità di rilassamento**:

1. **Extent**: si rilassa l'insieme di tuple su cui la dipendenza deve valere (eccezioni, condizioni, probabilità).
2. **Confronto**: si rilassa il modo in cui vengono confrontati i valori (similarità, distanze, matching).

- Le RFD trovano applicazione in **Data Cleaning**, **Data Quality**, **Entity Resolution**, **Record Matching**, **Data Integration**, **Query Optimization** e **Data Mining**.

Cos'è la Data Integration?

Nella pratica, i dati vengono spesso creati indipendentemente da diversi sistemi informativi. Successivamente nasce la necessità di combinarli. La **Data Integration** è quindi il processo che integra dati provenienti da sorgenti:

- multiple;
- distribuite;
- autonome;
- eterogenee;

per offrire un'unica vista dei dati agli utenti. L'idea è quella di nascondere la complessità delle sorgenti tramite uno **schema globale (mediated schema)** sul quale gli utenti eseguono le query, mentre il sistema si occupa di recuperare i dati dalle varie sorgenti.

Obiettivi

Una soluzione di Data Integration deve permettere di lavorare con dati:

- provenienti da sorgenti diverse;
- distribuite su reti differenti;
- gestite autonomamente;
- costruite con modelli differenti.

L'utente deve poter interrogare tutto tramite un'unica interfaccia, senza conoscere l'organizzazione delle sorgenti originali.

Problematiche principali

L'integrazione presenta difficoltà su tre livelli.

Livello di sistema

Problemi legati all'infrastruttura:

- DBMS differenti;
- piattaforme differenti;
- query distribuite.

Livello logico

Problemi dovuti alle differenze tra gli schemi e tra i dati.

Livello sociale

Problemi organizzativi:

- individuare i dati realmente utili;
- convincere le organizzazioni a condividerli;
- privacy;
- sicurezza.

Eterogeneità degli schemi

Gli schemi possono essere diversi pur rappresentando lo stesso dominio.
Le principali cause sono:

- diversità di prospettiva;
- equivalenza dei costrutti;
- incompatibilità delle specifiche;
- concetti comuni;
- concetti correlati.

Diversità di prospettiva

Due progettisti possono descrivere lo stesso mondo reale da punti di vista differenti.

Equivalenza dei costrutti

Uno stesso concetto può essere rappresentato in modi diversi.

Incompatibilità delle specifiche

Due modelli possono descrivere la stessa realtà imponendo regole incompatibili.

Relazioni tra rappresentazioni

Due rappresentazioni possono essere:

- **Identiche** → stesso schema.
- **Equivalenti** → schema diverso ma stesso significato.

- **Comparabili** → differiscono ma non si contraddicono.
- **Incompatibili** → descrivono realtà in conflitto.

Concetti correlati

Quando si uniscono più schemi possono emergere nuove relazioni che prima non erano visibili.
Queste vengono chiamate **proprietà interschema.**

Eterogeneità dei dati

Non cambiano solo gli schemi, ma anche il significato dei dati.

Architetture di integrazione

Esistono due strategie principali.

1. **Database materializzato (Data Warehouse)**

I dati vengono copiati periodicamente in un nuovo database centrale mediante processi **ETL (Extract-Transform-Load)**.

Vantaggi

- query molto veloci;
- ottime prestazioni;
- ideale per analisi e OLAP.

Svantaggi

- i dati potrebbero non essere aggiornati;
- richiede spazio aggiuntivo.

2. **Database virtuale (Vista Virtuale)**

I dati rimangono nelle sorgenti originali.
Quando arriva una query:

1. viene tradotta;
2. inviata alle sorgenti;
3. i risultati vengono combinati.

Sono necessari dei **wrapper**, che traducono query e risultati tra schema globale e sorgenti locali.

Vantaggi

- dati sempre aggiornati;
- nessuna duplicazione.

Svantaggi

- query più lente;
- traduzione delle query complessa.

Integrazione degli schemi

L'integrazione logica avviene in tre fasi:

1. identificazione dei concetti comuni;
2. analisi e risoluzione dei conflitti;
3. costruzione dello schema integrato.

Schema Matching

Consiste nell'individuare quali elementi appartenenti a schemi diversi rappresentano lo stesso concetto.

Analisi dei conflitti

Durante l'integrazione si possono incontrare quattro categorie di conflitti.

Conflitti di eterogeneità

Dipendono dall'utilizzo di modelli diversi.

Conflitti semantici

La stessa realtà è descritta con livelli di dettaglio differenti.

Conflitti sui nomi

Possono verificarsi:

1. Sinonimia: Nomi diversi per lo stesso concetto.
2. Omonimia: Lo stesso nome indica concetti diversi.

Conflitti strutturali

Sono dovuti a differenti scelte di modellazione.
Possono riguardare:

- tipo di dato;
- dipendenze;
- chiavi;
- comportamento delle operazioni di modifica e cancellazione.

Produzione dello schema integrato

Alla fine dell'integrazione si ottiene:

- un nuovo schema concettuale;
- la stessa semantica (per quanto possibile) degli schemi originali;
- un insieme di **mapping** tra schema globale e sorgenti.

Mapping

Il mapping collega ogni elemento dello schema globale ai dati reali presenti nelle sorgenti. Esistono due strategie.

1. **GAV (Global As View)**

Lo schema globale viene definito come insieme di viste costruite sulle sorgenti.

Vantaggi

- interrogazioni semplici;
- query facili da espandere (*unfolding*).

Svantaggi

- aggiungere una nuova sorgente richiede modifiche allo schema globale;
- scarsa estensibilità.

2. **LAV (Local As View)**

Le sorgenti vengono invece descritte come viste dello schema globale.

Vantaggi

- molto più estensibile;
- facile aggiungere nuove sorgenti;
- manutenzione semplificata.

Svantaggi

- la riformulazione delle query (*query rewriting*) è molto più complessa.

Il problema della similarità

Il problema non consiste nel verificare se due documenti siano identici (operazione semplice), ma nel capire se siano **quasi uguali**, pur presentando modifiche, riordini o piccole variazioni.

Le tre tecniche fondamentali

Per trovare documenti simili vengono utilizzate tre tecniche che lavorano in sequenza:

1. Shingling

Trasforma ogni documento in un insieme di elementi (shingles);

2. Minhashing

Comprime questi insiemi in firme (signature) molto più piccole;

3. Locality Sensitive Hashing (LSH)

Evita di confrontare tutte le coppie possibili confrontando solo quelle probabilmente simili.

Shingling

Lo **shingling** rappresenta un documento come un insieme di sottostringhe di lunghezza fissa **k**, chiamate **k-shingles**. Due documenti saranno simili se condividono molti shingles.
La scelta di **k** è molto importante.
Se k è troppo piccolo, tantissimi falsi positivi.
Se k è troppo grande, molti falsi negativi.

Hashing degli shingles

Salvare ogni shingle come stringa occupa molta memoria. Per questo motivo si applica una funzione di hash. Il documento diventa quindi un insieme di interi (token). Le collisioni sono possibili ma molto rare.

Similarità di Jaccard

Una volta trasformati i documenti in insiemi, la misura più naturale è la **Jaccard Similarity**.

Formula: $J(A,B) = \frac{|A \cap B|}{|A \cup B|}$

La similarità varia tra:

- 0 → completamente diversi
- 1 → identici.

Minhashing

Gli insiemi di shingles possono essere enormi.
L'idea del **Minhash** è costruire una firma molto più corta mantenendo una buona stima della similarità.
Per farlo:

1. si immagina di permutare casualmente le righe della matrice caratteristica;
2. per ogni colonna si prende la **prima riga contenente 1**.

Questo valore è il **minhash**.
Ripetendo il procedimento con molte permutazioni casuali si ottiene una firma.

Se due documenti sono molto simili, è molto probabile che abbiano gli stessi minimi per molte funzioni hash, quindi le loro signature saranno simili. Infatti, una proprietà fondamentale del MinHash è: $P(h(A) = h(B)) = Jaccard(A,B)$ cioè la probabilità che due documenti abbiano lo stesso valore MinHash per una funzione hash è esattamente uguale alla loro similarità di Jaccard. Questo è il motivo per cui confrontare le signature permette di stimare la similarità tra documenti senza confrontare tutti gli shingles.

Maggiore è la lunghezza della signature, più precisa sarà la stima

Implementazione del Minhash

Permutare realmente miliardi di righe è impossibile. La soluzione consiste nel sostituire le permutazioni con numerose funzioni hash casuali.
Per ogni funzione hash:

- si calcola il valore hash della riga;
- per ogni documento si mantiene il minimo valore hash osservato.

In questo modo si ottiene lo stesso effetto senza memorizzare enormi permutazioni.

Locality Sensitive Hashing (LSH)

Anche confrontare tutte le signature è troppo costoso. Lo LSH riduce drasticamente il numero di confronti.

Come funziona

La signature viene divisa in:

- **b bande (bands)**
- ciascuna composta da **r righe**

Per ogni banda:

- si applica un hash;
- documenti che finiscono nello stesso bucket diventano **candidate pair**.

Solo queste coppie verranno confrontate.

Probabilità di diventare candidati

Se:

- b = numero di bande
- r = righe per banda
- s = similarità

allora: $P(candidate) = 1 - \left(1 - s^r\right)^b$
Questa funzione produce la caratteristica **curva a S**:

- documenti molto simili vengono quasi sempre confrontati;

- documenti poco simili vengono quasi sempre scartati.

Il compromesso è tra accuratezza ed efficienza.
Aumentare **b** (più bande):
riduce i falsi negativi; aumenta i falsi positivi.
Aumentare **r** (più righe per banda)

Misure di distanza

`Distanza Euclidea (L2)`
È la distanza geometrica classica tra due punti.
`Distanza Manhattan (L1)`
Somma delle differenze assolute. È la distanza percorsa muovendosi lungo una griglia.
`Distanza di Minkowski`
Generalizza Euclidea e Manhattan.
`Distanza di Jaccard`
$1 - Jaccard$
Utilizzata per insiemi.
`Distanza Coseno`
Misura l'angolo tra due vettori.
Molto usata nel text mining.
`Edit Distance`
Numero minimo di inserimenti e cancellazioni necessari per trasformare una stringa in un'altra.
`Hamming Distance`
Numero di posizioni differenti tra due vettori della stessa lunghezza.

- **Shingling**: trasforma un documento in un insieme di k-shingles.
- **Jaccard Similarity**: misura la similarità tra due insiemi come intersezione/unione.
- **Minhash**: comprime gli insiemi in signature preservando la similarità.
- **Proprietà fondamentale**: P(h(A)=h(B))=J(A,B)P(h(A)=h(B)) = J(A,B)P(h(A)=h(B))=J(A,B).
- **LSH**: evita confronti esaustivi confrontando solo coppie candidate.
- **Formula LSH**: P=1−(1−sr)bP=1-(1-s^r)^bP=1−(1−sr)b.
- La scelta dei parametri **k**, **b** e **r** determina il compromesso tra accuratezza, falsi positivi, falsi negativi ed efficienza.
- Le principali metriche di distanza da conoscere sono: Euclidea, Manhattan, Minkowski, Jaccard, Coseno, Edit Distance e Hamming Distance.

Cos'è il Data Mining

Il **Data Mining** è il processo di estrazione automatica di informazioni dai dati esistenti, con l'obiettivo di trovare pattern che siano:

- **validi**, cioè verificabili anche su nuovi dati;
- **precedentemente sconosciuti**;
- **potenzialmente utili**;
- **comprensibili** per l'utente.

L'idea è trasformare grandi quantità di dati grezzi in conoscenza utile per prendere decisioni.

Knowledge Discovery (KD)

Il Data Mining rappresenta la fase centrale del processo di **Knowledge Discovery (KD)**.
Gli elementi fondamentali sono:

- **Dati**: informazioni contenute in database o data warehouse.
- **Pattern**: rappresentazioni sintetiche delle informazioni estratte, cioè regolarità presenti nei dati.
- **Conoscenza**: interpretazione dei pattern da parte dell'utente.

Ad esempio, osservando i dati relativi ai prestiti bancari si può scoprire una regola del tipo:
Se lo stipendio è inferiore ad una certa soglia k, allora aumenta la probabilità di mancati pagamenti.
Questa regola rappresenta un pattern estratto automaticamente dai dati.

Caratteristiche di un buon pattern

Non tutte le regole trovate sono interessanti.
Un pattern deve soddisfare quattro proprietà fondamentali.
`Validità`
Deve continuare a funzionare anche su dati futuri.
`Novità`
Deve fornire informazioni che non erano già note.
`Utilità`
Deve produrre un vantaggio concreto.
`Comprensibilità`
Il pattern deve essere facilmente interpretabile dalle persone.

Il processo di Data Mining

L'estrazione della conoscenza avviene attraverso diverse fasi:
$Dati \rightarrow Selezione \rightarrow Preprocessing \rightarrow Trasformazione \rightarrow Data\ Mining \rightarrow Pattern \rightarrow Conoscenza$
Durante questo processo è importante che gli algoritmi siano:

- efficienti;
- scalabili su basi di dati molto grandi;
- capaci di lavorare con dati eterogenei;
- in grado di valutare la qualità dei pattern trovati.

Regole di associazione

Uno dei problemi più famosi del Data Mining consiste nello scoprire prodotti acquistati frequentemente insieme (**Market Basket Analysis**).
Si definiscono:

- **Item**: un prodotto.
- **Transazione**: insieme dei prodotti acquistati contemporaneamente.
- **Database**: insieme di tutte le transazioni.

Una regola di associazione ha la forma: $X \Rightarrow Y$
dove:

- **X** è l'insieme degli antecedenti;
- **Y** è l'insieme dei conseguenti.

Misure di qualità

Le regole vengono valutate attraverso due misure.

Supporto

Il supporto misura **quanto frequentemente compare una regola**.
Formalmente:
$Supporto(X \Rightarrow Y) = Supporto(X \cup Y)$
cioè il numero (o la percentuale) di transazioni che contengono sia X che Y.

Confidenza

La confidenza misura **quanto è affidabile l'implicazione**.
Formula:
$Confidenza(X \Rightarrow Y) = \frac{Supporto(X \cup Y)}{Supporto(X)}$
È la probabilità che venga acquistato Y sapendo che è stato acquistato X.

Il problema delle regole di associazione

L'obiettivo è trovare **tutte le regole** che rispettano due soglie fissate dall'utente:

- supporto minimo;
- confidenza minima.

Frequent Itemset

Per evitare di generare tutte le possibili regole (numero enorme), il problema viene diviso in due fasi.

Fase 1

Trovare tutti gli **itemset frequenti**.
Un itemset è frequente se il suo supporto supera la soglia minima.

Fase 2

Generare le regole soltanto a partire dagli itemset frequenti.

Algoritmo Apriori

L'algoritmo Apriori è il metodo classico per trovare i frequent itemset.
Lavora in modo iterativo.

Passo 1

Conta il supporto dei singoli item.
Ottiene: $L1$ cioè gli item frequenti.

Passo 2

Combina gli item frequenti a due a due.
Ottiene: $C2$ (candidate itemset).
Conta il supporto. Elimina quelli non frequenti.
Ottiene: $L2$

Passo 3

Combina gli itemset di dimensione due.
Produce: $C3$
Elimina quelli non frequenti.
Ottiene: $L3$
Il procedimento continua finché non esistono più itemset frequenti.

Proprietà di Apriori

Il principio fondamentale è:
Ogni sottoinsieme di un frequent itemset è anch'esso un frequent itemset.
Da questa proprietà deriva una forte ottimizzazione. Se un sottoinsieme non è frequente, nessun insieme più grande che lo contiene potrà esserlo.
Di conseguenza molti candidati vengono eliminati senza essere controllati sul database.

Generazione delle regole

Una volta trovati i frequent itemset, si costruiscono le regole.
Per ogni frequent itemset: X lo si divide in:
$LHS \rightarrow RHS$ e si calcola $Confidenza = \frac{Supporto(X)}{Supporto(LHS)}$
Poiché il supporto di tutti i sottoinsiemi è già stato calcolato nella prima fase, questa operazione risulta molto veloce.

Gerarchie tra item

In alcuni casi gli oggetti sono organizzati gerarchicamente.
Sostituendo un prodotto con una categoria più generale, il supporto può solo aumentare.
Questo permette di estrarre regole anche a livelli di astrazione differenti.

Estensioni

Regole condizionate

Le regole vengono cercate solo su un sottoinsieme dei dati.

Pattern sequenziali

Le transazioni di uno stesso cliente vengono ordinate temporalmente.
L'obiettivo non è più trovare prodotti acquistati insieme, ma sequenze frequenti di acquisti nel tempo.

- Il **Data Mining** estrae conoscenza utile dai dati.
- Un buon pattern deve essere **valido, nuovo, utile e comprensibile**.
- Le **regole di associazione** descrivono relazioni tra prodotti acquistati insieme.
- Il **supporto** misura la frequenza della regola.
- La **confidenza** misura quanto è affidabile l'implicazione.
- L'algoritmo **Apriori** trova prima gli **itemset frequenti**, poi genera le regole.
- La proprietà fondamentale di Apriori è che **ogni sottoinsieme di un itemset frequente è anch'esso frequente**, consentendo di eliminare molti candidati e ridurre drasticamente il numero di controlli sul database.

Classificazione e regressione

La **classificazione** consiste nell'assegnare un oggetto ad una classe nota utilizzando esempi già etichettati (training set).
Sono necessari:

- un insieme di classi;
- un training set contenente esempi già classificati;
- un test set costituito da dati mai visti durante l'addestramento.

L'obiettivo è imparare una funzione che descriva ogni classe e che sia capace di classificare correttamente nuovi esempi.
La **regressione**, invece, predice un valore numerico anziché una categoria.
La differenza principale è quindi:

- **Classificazione → output categorico**
- **Regressione → output numerico**

Training set e Test set

Uno degli aspetti fondamentali del Machine Learning è la distinzione tra dati di addestramento e dati di test. Il motivo della divisione è semplice: interessa valutare quanto il modello **generalizza**, cioè quanto è capace di classificare dati mai osservati. Non interessa ottenere il 100% sul training set, perché un modello potrebbe semplicemente aver memorizzato gli esempi (**overfitting**).
Generalmente il dataset viene diviso in:

- **70–80% Training Set**
- **20–30% Test Set**

Il training set serve a costruire il modello.
Il test set serve esclusivamente a misurarne le prestazioni.

Regole di classificazione

Una regola di classificazione ha la forma
$P_1(X_1) \wedge P_2(X_2) \wedge \ldots \wedge P_k(X_k) \Rightarrow Y = c$
dove:

- X_i sono gli attributi predittivi (feature);
- Y è la classe da prevedere;
- P_i rappresenta una condizione sugli attributi.

Le feature possono essere:

`Numeriche`

La condizione assume la forma $l_i \leq X_i \leq g_i$

`Categoriche`

La condizione consiste nell'appartenenza ad un insieme

Supporto e confidenza

Come nelle regole associative, anche qui esistono due misure.

`Support`

Percentuale di record che soddisfano contemporaneamente antecedente e conseguente.

`Confidence`

Probabilità che la conclusione sia vera quando sono vere le condizioni iniziali. Una confidence elevata indica una regola molto affidabile.

Tipologie di classificatori

Esistono numerosi algoritmi di classificazione.
Tra i principali:

- Decision Tree
- Bayesian Classifiers
- Statistical Classifiers
- Neural Networks
- k-NN
- Support Vector Machine

Decision Tree

Un **Decision Tree** rappresenta le regole di classificazione mediante un albero.
Ogni nodo interno rappresenta una domanda.
Ogni foglia rappresenta una classe.
Vantaggi:

- molto veloce;
- facilmente interpretabile;
- facilmente traducibile in regole SQL;
- ogni percorso dalla radice ad una foglia corrisponde ad una regola.

Algoritmo k-Nearest Neighbors (k-NN)

Il **k-NN** è uno degli algoritmi più semplici del Machine Learning.
È un algoritmo **instance-based** o **lazy learning**, perché non costruisce un vero modello durante l'addestramento: conserva semplicemente i dati di training. Quando arriva un nuovo esempio:

1. calcola la distanza da tutti gli esempi;
2. individua i **k vicini più vicini**;
3. assegna la classe più frequente.

Se $k = 1$ l'oggetto assume direttamente la classe del vicino più prossimo. Occorre scegliere:

- il valore di **k**
- una misura di distanza (Euclidea, Manhattan, ecc.).

`Scelta di k`

Un valore piccolo:

- segue molto i dati;
- è sensibile al rumore.

Un valore grande:

- riduce il rumore;
- rende meno netti i confini tra le classi.

Il valore ottimale viene spesso trovato mediante **Cross Validation**.

Valutazione dei classificatori

Un buon classificatore dovrebbe possedere:

- elevata accuratezza;
- velocità;
- robustezza;
- scalabilità;
- facilità di interpretazione;
- trasparenza.

Valutazione del modello

La procedura classica è:

1. dividere il dataset;
2. addestrare il modello con `fit()`;
3. valutare il modello con `score()` sul test set.

Lo score rappresenta la percentuale di esempi classificati correttamente.

Cross Validation

La **Cross Validation** è una tecnica più affidabile del semplice train/test split.
Nel metodo **k-fold**:

1. il dataset viene diviso in k parti;
2. ogni parte diventa una volta test set;
3. le altre k−1 parti diventano training set;
4. si ripete il processo k volte;
5. si calcola la media delle prestazioni.

`Vantaggi`

- valutazione più stabile;
- sfrutta meglio i dati;
- riduce la dipendenza dalla suddivisione casuale.

`Svantaggio`

Bisogna addestrare il modello k volte, quindi aumenta il tempo computazionale.

Confusion Matrix

La **Confusion Matrix** descrive completamente il comportamento di un classificatore binario.
Contiene quattro valori.

`True Positive (TP)`

`True Negative (TN)`

`False Positive (FP)`

`False Negative (FN)`

La diagonale principale contiene le classificazioni corrette.

`Accuracy`

L'Accuracy misura la percentuale di classificazioni corrette. $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$

È semplice ma può essere fuorviante quando le classi sono sbilanciate.

`Precision`

La Precision risponde alla domanda: Tra tutti gli elementi classificati come positivi, quanti lo sono davvero? $Precision = \frac{TP}{TP+FP}$

È importante quando i **False Positive** sono molto costosi.

`Recall`

La Recall risponde alla domanda: Quanti positivi reali siamo riusciti a trovare? $Recall = \frac{TP}{TP+FN}$

È fondamentale quando bisogna evitare i **False Negative**.

Trade-off Precision/Recall

Precision e Recall sono in competizione. Aumentare una, tende a diminuire l'altra.

F1-score

Per combinare Precision e Recall si usa l'**F1-score**, che è la loro media armonica.

$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision+Recall}$

Un valore elevato indica un buon equilibrio tra Precision e Recall.

ROC Curve

La **ROC Curve** rappresenta:

- sull'asse x → False Positive Rate (FPR);
- sull'asse y → True Positive Rate (Recall).

Un buon classificatore produce una curva vicina all'angolo superiore sinistro. La diagonale rappresenta un classificatore casuale.

ROC o Precision-Recall?

- **ROC**: preferibile quando le classi sono bilanciate.
- **Precision-Recall**: preferibile quando la classe positiva è rara o i falsi positivi sono particolarmente critici.

Classificatore Naïve Bayes

Naïve Bayes è un classificatore probabilistico.
Per ogni classe calcola la probabilità che il nuovo esempio appartenga a quella classe e sceglie quella con probabilità maggiore.

Teorema di Bayes

$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$ dove:

- **Prior** $P(A)$: probabilità iniziale della classe;
- **Likelihood** $P(B|A)$: probabilità di osservare i dati assumendo la classe;
- **Evidence** $P(B)$: probabilità complessiva dei dati osservati;
- **Posterior** $P(A|B)$: probabilità aggiornata della classe dopo aver osservato i dati.

Ipotesi "Naïve"

Il problema principale è che calcolare la probabilità con molte feature è complesso.
Naïve Bayes assume che tutte le feature siano **indipendenti** tra loro. Per questo:
$P(x|y) = P(x_1|y)P(x_2|y)\dots(x_n|y)$
Questa ipotesi semplifica enormemente il calcolo e rende l'algoritmo molto veloce.

Laplace Smoothing

Può accadere che una probabilità osservata nel training set sia pari a zero. Moltiplicando per zero tutta la probabilità diventerebbe zero. Per evitarlo si usa il **Laplace Smoothing**, aggiungendo un piccolo valore (tipicamente 1) ai conteggi. In questo modo nessuna probabilità risulta nulla.

Tipi di classificazione

Binary Classification

Due sole classi.

Multiclass Classification

Più di due classi.
Alcuni algoritmi la supportano direttamente (Random Forest, Naïve Bayes). Altri usano strategie dedicate.

OvA (One vs All)

Si costruisce un classificatore per ogni classe contro tutte le altre.

OvO (One vs One)

Si costruisce un classificatore per ogni coppia di classi. Con N classi servono:
$\frac{N(N-1)}{2}$ classificatori. È spesso usato con le SVM.

Multilabel Classification

Un esempio può appartenere contemporaneamente a più classi.

Multioutput Classification

Ogni output può assumere più valori.

- La classificazione predice **classi**, la regressione **valori numerici**.
- Il dataset va sempre diviso in **training** e **test**.
- **k-NN** classifica usando i **k vicini più vicini**.
- La **Cross Validation** fornisce una stima più affidabile delle prestazioni rispetto a un singolo train/test split.
- La **Confusion Matrix** è la base per calcolare Accuracy, Precision, Recall e F1-score.
- **Precision** è importante quando si vogliono ridurre i falsi positivi, **Recall** quando si vogliono ridurre i falsi negativi.
- L'**F1-score** sintetizza Precision e Recall in un'unica misura.
- La **ROC Curve** valuta il compromesso tra veri positivi e falsi positivi.
- **Naïve Bayes** utilizza il Teorema di Bayes assumendo l'indipendenza tra le feature, semplificando il calcolo delle probabilità e risultando particolarmente efficiente.

Il problema del clustering

Il clustering consiste nel dividere un insieme di punti in gruppi (**cluster**) tali che:

- i punti appartenenti allo stesso cluster siano molto simili tra loro;
- i punti appartenenti a cluster diversi siano il più possibile differenti.

Per definire la similarità si utilizza una **misura di distanza**, ad esempio:

- distanza euclidea;
- distanza coseno;
- distanza di Jaccard;
- edit distance.

Nella maggior parte delle applicazioni i dati vivono in **spazi ad alta dimensionalità**, dove ogni punto possiede decine, centinaia o migliaia di caratteristiche.

Perché il clustering è difficile?

In due dimensioni il clustering sembra semplice, ma nella realtà:

- i dati hanno spesso migliaia di dimensioni;
- nello spazio ad alta dimensionalità quasi tutte le coppie di punti risultano avere distanze molto simili.

Questo fenomeno rende molto più difficile distinguere chiaramente i cluster.

Metodi di clustering

La lezione distingue due grandi famiglie di algoritmi.

Clustering gerarchico (Hierarchical Clustering)

Costruisce una gerarchia di cluster.

Può essere:

Agglomerative (bottom-up)

- ogni punto rappresenta inizialmente un cluster;
- ad ogni iterazione vengono uniti i due cluster più vicini;
- il processo continua fino a ottenere il numero desiderato di cluster.

Divisive (top-down)

Parte da un unico cluster contenente tutti i punti e lo divide progressivamente.

Problemi principali

Ogni algoritmo gerarchico deve risolvere tre domande:

1. come rappresentare un cluster;
2. come misurare la distanza tra due cluster;
3. quando interrompere le fusioni.

Centroid e Clustroid

Centroid

Nel caso euclideo ogni cluster viene rappresentato dal **centroide**, cioè la media di tutti i punti del cluster. Il centroide:

- non deve necessariamente coincidere con un punto reale;
- rappresenta il "centro geometrico" del cluster.

Clustroid

Quando non è possibile calcolare una media (ad esempio con dati non euclidei), si utilizza il **clustroid**. Il clustroid è:

- un punto realmente esistente;
- quello più vicino, nel complesso, a tutti gli altri punti del cluster.

Quindi:

- **centroid = punto artificiale**
- **clustroid = punto reale**

Come misurare la distanza tra cluster

Esistono diversi criteri.

Distanza tra centroidi

Si confrontano i centroidi dei cluster.

Single Link

Si considera la distanza minima tra due punti appartenenti ai due cluster.

Coesione

Si misura quanto il cluster risultante sia compatto.

Possibili misure di coesione:

- diametro del cluster;
- distanza media tra i punti;
- densità del cluster.

Complessità del clustering gerarchico

L'implementazione più semplice richiede: $O(N^3)$

Una versione ottimizzata con code di priorità arriva a: $O(N^2 logN)$

Rimane comunque troppo costosa per dataset molto grandi.

Algoritmo K-Means

K-Means è probabilmente l'algoritmo di clustering più utilizzato. Assume che:

- lo spazio sia euclideo;
- il numero di cluster **k** sia noto.

Fasi dell'algoritmo

1. Scelta di k

Si decide quanti cluster cercare.

2. Inizializzazione

Si scelgono i centroidi iniziali. Una strategia comune consiste nello scegliere un punto casuale e poi gli altri il più lontano possibile da quelli già scelti.

3. Assegnazione

Ogni punto viene assegnato al centroide più vicino.

4. Aggiornamento

Si ricalcolano i centroidi come media dei punti assegnati.

5. Ripetizione

Le ultime due fasi vengono ripetute finché:

- i centroidi non cambiano più;
- nessun punto cambia cluster.

Questo è il criterio di convergenza.

Come scegliere k

Il numero ottimale di cluster viene spesso individuato con il **metodo del gomito (Elbow Method)**. Si osserva l'andamento della distanza media dal centroide:

- inizialmente diminuisce molto rapidamente;
- oltre un certo valore di k il miglioramento diventa minimo.

Il "gomito" della curva rappresenta il valore migliore di k.

Algoritmo BFR

Il **BFR (Bradley-Fayyad-Reina)** è un'estensione di K-Means progettata per dataset troppo grandi per essere caricati completamente in memoria.
Assume che:

- i cluster abbiano distribuzione normale;
- abbiano forma ellittica allineata agli assi;
- sia disponibile la distanza euclidea.

`Idea principale`

Il dataset viene letto **a blocchi** dal disco.
Non vengono memorizzati tutti i punti, ma solo statistiche riassuntive dei cluster.

`Le tre categorie di punti`

`Discard Set (DS)`

Contiene i punti già assegnati con sicurezza a un cluster. Di essi si conservano solo statistiche.

`Compression Set (CS)`

Contiene piccoli gruppi di punti vicini (minicluster) che non possono ancora essere assegnati definitivamente.

`Retained Set (RS)`

Contiene punti isolati che non appartengono ancora ad alcun cluster.

`Riassunto statistico dei cluster`

Ogni cluster viene rappresentato tramite:

- **N** = numero di punti;
- **SUM** = somma delle coordinate;
- **SUMSQ** = somma dei quadrati delle coordinate.

Da queste statistiche è possibile ricavare:

- centroide;
- varianza;
- deviazione standard,

senza memorizzare tutti i punti. Lo spazio richiesto cresce con il numero di cluster e non con il numero totale di dati.

`Elaborazione di ogni blocco`

Per ogni nuovo blocco di dati:

1. i punti vicini ai centroidi vengono aggiunti al DS;
2. i punti rimanenti vengono raggruppati;
3. si aggiornano le statistiche;
4. si fondono eventuali minicluster del CS;
5. al termine tutti i punti vengono assegnati ai cluster oppure considerati outlier.

Distanza di Mahalanobis

Per decidere se un punto appartiene a un cluster, BFR utilizza la **distanza di Mahalanobis**.
A differenza della distanza euclidea, ogni coordinata viene normalizzata rispetto alla deviazione standard del cluster:

$$d(x,c) = \sqrt{\sum_i \left(\frac{x_i - c_i}{\sigma_i}\right)^2}$$

Questa distanza tiene conto della dispersione dei dati. Se la distanza è inferiore a una soglia (ad esempio **4 deviazioni standard**), il punto viene assegnato al cluster con probabilità molto elevata.

Algoritmo CURE

Il **CURE (Clustering Using REpresentatives)** supera i limiti di K-Means e BFR.
Infatti questi ultimi assumono cluster:

- sferici o ellittici;
- distribuiti normalmente.

CURE permette invece cluster di forma arbitraria.

`Funzionamento`

`Prima fase`

- si estrae un campione casuale;
- si applica clustering gerarchico;
- per ogni cluster vengono scelti alcuni punti molto dispersi;
- tali punti vengono spostati di circa il **20% verso il centroide**.

Questi diventano i rappresentanti del cluster.

`Seconda fase`

Si rilegge tutto il dataset.
Ogni nuovo punto viene assegnato al cluster il cui rappresentante risulta più vicino.

Confronto tra gli algoritmi

Algoritmo	Vantaggi	Svantaggi
Hierarchical Clustering	Non richiede di fissare inizialmente i cluster, produce un dendrogramma	Molto costoso su grandi dataset
K-Means	Semplice, veloce, efficiente	Richiede il numero di cluster, funziona bene solo con cluster sferici
BFR	Adatto a dataset enormi, usa poca memoria	Assume distribuzioni gaussiane e cluster ellittici
CURE	Gestisce cluster di forma arbitraria	Più complesso e computazionalmente costoso

- Il **clustering** è una tecnica di apprendimento non supervisionato che raggruppa dati simili.
- Le principali misure di similarità sono **Euclidea, Coseno e Jaccard**.
- Il **clustering gerarchico** costruisce una gerarchia di cluster tramite fusioni o divisioni.
- Il **centroide** è la media dei punti, mentre il **clustroid** è un punto reale rappresentativo del cluster.
- **K-Means** alterna assegnazione dei punti e aggiornamento dei centroidi fino alla convergenza.
- Il valore di **k** viene spesso scelto con il **metodo del gomito**.
- **BFR** estende K-Means ai Big Data grazie ai riassunti statistici (**N, SUM, SUMSQ**) e alla distanza di **Mahalanobis**.
- **CURE** rappresenta ogni cluster con più punti rappresentativi, riuscendo a identificare cluster di forma arbitraria.

Addestramento dei modelli

L'obiettivo dell'addestramento (**training**) è trovare i parametri del modello che minimizzano una funzione di costo (cost function).

- **Normal Equation**, che calcola direttamente la soluzione ottimale tramite una formula matematica.
- **Gradient Descent**, che trova la soluzione in modo iterativo migliorando progressivamente i parametri.

Regressione Lineare

La **Linear Regression** è uno dei modelli più semplici del Machine Learning.
Predice il valore dell'output come somma pesata delle feature: $\widehat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots + \theta_n x_n$

dove:

- x_i = feature
- θ_i = peso della feature
- θ_0 = bias (intercetta)

In forma vettoriale: $\widehat{y} = h_\theta(x) = \theta^T x$

Questa rappresentazione è molto utilizzata perché permette implementazioni efficienti tramite algebra lineare.

Normal Equation

La **Normal Equation** permette di trovare direttamente i parametri ottimali: $\widehat{\theta} = \left(X^T X\right)^{-1} X^T y$
Non richiede iterazioni.

`Vantaggi`

- soluzione esatta;
- semplice per dataset piccoli.

`Svantaggi`

Richiede l'inversione di una matrice.

La complessità cresce tra $O\left(n^{2.4}\right)$ e $O\left(n^3\right)$ dove n è il numero di feature. Per moltissime feature (ad esempio 100.000) diventa troppo costosa.
Le predizioni invece rimangono molto veloci (complessità lineare).

Gradient Descent

Quando la Normal Equation diventa impraticabile si usa il **Gradient Descent (GD)**.
È un algoritmo di ottimizzazione iterativo.
L'idea è semplice:

1. inizializzare casualmente i parametri;
2. calcolare il gradiente della funzione di costo;
3. spostarsi nella direzione opposta al gradiente;
4. ripetere finché non si raggiunge un minimo.

`Learning Rate`

Il parametro fondamentale è il **learning rate** (η).
Se è troppo piccolo, convergenza molto lenta.
Se è troppo grande, il minimo viene superato continuamente; l'algoritmo può divergere.
Occorre scegliere un valore equilibrato.

`Minimi locali`

In generale una funzione di costo può avere:

- minimi locali;
- plateau;
- regioni molto piatte.

Per fortuna, nella regressione lineare con errore MSE, la funzione è **convessa**.
Questo significa che esiste un solo minimo globale.
Di conseguenza, Gradient Descent è garantito a convergere (con learning rate adeguato).

`Feature Scaling`

Prima di applicare Gradient Descent conviene normalizzare le feature.
Se una feature ha valori molto più grandi delle altre:

- il "paesaggio" della funzione di costo diventa molto allungato;
- Gradient Descent procede lentamente.

Con feature scalate, il minimo viene raggiunto molto più rapidamente.

Con feature scalate, il minimo viene raggiunto molto più rapidamente.

Polynomial Regression

La regressione lineare può essere estesa a dati non lineari aggiungendo nuove feature ottenute elevando le feature originali a potenze.
Il modello rimane lineare nei parametri, ma può rappresentare curve complesse.
Questa tecnica prende il nome di **Polynomial Regression**.

Complessità del modello

Il grado del polinomio determina la complessità.

Grado troppo basso: Underfitting

Il modello è troppo semplice.

Grado troppo alto: Overfitting

Il modello segue perfettamente il training set ma generalizza male.
Il miglior modello è quello sufficientemente complesso da descrivere i dati senza adattarsi al rumore.

Learning Curves

Le **Learning Curves** mostrano come varia l'errore:

- training set;
- validation set;

all'aumentare della quantità di dati di addestramento. Servono per capire se il modello: underfitta odoverfitta.

Underfitting

Nel caso di underfitting:

- errore di training alto;
- errore di validazione alto;
- le due curve sono vicine.

Aggiungere altri dati non migliora la situazione.
Occorre:

- usare un modello più complesso;
- costruire feature migliori.

Bias-Variance Tradeoff

L'errore di generalizzazione è fatto di tre parti.

Bias

Errore dovuto a ipotesi troppo semplici.
Produce underfitting

Variance

Errore dovuto a modelli troppo complessi.
Produce overfitting

Errore irriducibile

Dipende dal rumore nei dati. Può essere ridotto solo migliorando la qualità del dataset.

Esiste quindi un compromesso:

- aumentare la complessità ↓ bias ma ↑ variance;
- ridurre la complessità ↑ bias ma ↓ variance.

Regolarizzazione

Per ridurre l'overfitting si aggiunge un termine di penalizzazione alla funzione di costo.
L'obiettivo è limitare l'entità dei pesi del modello.

Ridge Regression

Aggiunge una penalizzazione $L2$:

$$J(\theta) = MSE + \frac{\alpha}{2}\sum \theta_i^2$$

Effetti:

- pesi piccoli;
- modello più stabile;
- riduzione dell'overfitting.

Il parametro α:

- piccolo → modello simile alla regressione lineare;
- grande → modello molto semplice.

È importante normalizzare le feature prima dell'addestramento.

Lasso Regression

Usa la penalizzazione $L1$:

$$J(\theta) = MSE + \alpha\sum\left|\theta_i\right|$$

Caratteristica principale: alcuni coefficienti diventano esattamente zero.
Questo equivale a fare automaticamente **feature selection**.

Elastic Net

Combina Ridge e Lasso:

- parte della penalizzazione è L1;
- parte è L2.

È spesso preferito perché:

- mantiene la selezione delle feature;
- è più stabile del Lasso quando le feature sono molto correlate.

Early Stopping

Un'altra tecnica di regolarizzazione consiste nel fermare l'addestramento quando l'errore sul validation set smette di diminuire.
Infatti:

- l'errore sul training continua a diminuire;
- quello sul validation, dopo un certo punto, aumenta.

Quel momento indica l'inizio dell'overfitting.

Logistic Regression

Nonostante il nome, è un algoritmo di **classificazione binaria**.
Calcola la probabilità che un'istanza appartenga alla classe positiva.
Utilizza la funzione sigmoide: $\sigma(t) = \frac{1}{1+e^{-t}}$
che restituisce valori compresi tra 0 e 1.
Se: $p \geq 0.5$ predice la classe 1.
Altrimenti predice la classe 0.

`Funzione di costo`

La Logistic Regression utilizza la **Log Loss (Cross Entropy)**:

- penalizza molto le probabilità errate;
- premia probabilità corrette vicine a 1.

Non esiste una formula chiusa come nella regressione lineare.
L'addestramento viene effettuato con Gradient Descent, sfruttando il fatto che la funzione di costo è convessa.

`Decision Boundary`

La **decision boundary** rappresenta il confine che separa le due classi.
Per due classi:

- probabilità = 50%;
- da una parte viene predetta la classe 0;
- dall'altra la classe 1.

Con più feature il confine diventa una linea (o un iperpiano) nello spazio delle feature.

Softmax Regression

La Logistic Regression può essere estesa alla classificazione multiclasse.
Questa estensione prende il nome di:

- **Softmax Regression**
- **Multinomial Logistic Regression**

Il modello:

1. calcola uno score per ogni classe;
2. applica la funzione Softmax;
3. ottiene una distribuzione di probabilità sulle classi;
4. sceglie la classe con probabilità massima (`argmax`).

È adatta solo a classi **mutuamente esclusive**, cioè un'istanza può appartenere a una sola classe. La funzione di costo utilizzata è la **Cross Entropy**, che penalizza le basse probabilità assegnate alla classe corretta.

- **Linear Regression** → predizione di valori continui.
- **Normal Equation** → soluzione esatta, ma poco scalabile.
- **Gradient Descent** → ottimizzazione iterativa per grandi dataset.
- **Learning Rate** → controlla la velocità di apprendimento.
- **Feature Scaling** → accelera la convergenza del Gradient Descent.
- **Polynomial Regression** → permette di modellare relazioni non lineari.
- **Learning Curves** → aiutano a riconoscere underfitting e overfitting.
- **Bias-Variance Tradeoff** → equilibrio tra semplicità e complessità del modello.
- **Ridge (L2)** → riduce i pesi senza eliminarli.
- **Lasso (L1)** → elimina automaticamente le feature meno importanti.
- **Elastic Net** → combina Ridge e Lasso.
- **Early Stopping** → interrompe l'addestramento prima dell'overfitting.
- **Logistic Regression** → classificazione binaria tramite funzione sigmoide.
- **Softmax Regression** → estende la Logistic Regression alla classificazione multiclasse.

Cos'è una Support Vector Machine

L'obiettivo principale di una SVM è trovare un **iperpiano di separazione** tra le classi che massimizzi la distanza dai punti più vicini appartenenti alle classi stesse.
L'idea fondamentale è:

- non basta separare correttamente le classi;
- bisogna separarle con il **margine più ampio possibile**.

Maggiore è il margine, migliore sarà in genere la capacità di generalizzazione del modello.

Large Margin Classification

Tra tutti gli iperpiani che separano perfettamente i dati, la SVM sceglie quello che lascia la distanza maggiore tra le due classi.
Si immagina una "strada" delimitata da due linee parallele:

- al centro c'è l'iperpiano decisionale;
- ai lati ci sono i margini.

L'obiettivo è rendere questa strada il più larga possibile. Solo alcuni punti determinano realmente la posizione dell'iperpiano. Questi punti prendono il nome di **Support Vectors**. Sono proprio loro a "sostenere" la soluzione della SVM. Tutti gli altri punti possono perfino essere rimossi senza modificare il classificatore.

Importanza della normalizzazione

Le SVM sono molto sensibili alla scala delle feature. Se una variabile assume valori molto maggiori delle altre, influenzerà eccessivamente il calcolo dell'iperpiano. Per questo motivo è pratica standard applicare uno **StandardScaler** prima dell'addestramento.

Hard Margin Classification

Nel caso ideale si richiede che:

- tutti i punti siano classificati correttamente;
- nessun punto entri nel margine.

Questa versione prende il nome di **Hard Margin SVM**.

Presenta però due grossi limiti:

- funziona solo con dati perfettamente separabili;
- è estremamente sensibile agli outlier.

Un singolo punto anomalo può cambiare completamente il classificatore oppure rendere impossibile trovare una soluzione.

Soft Margin Classification

Per risolvere questi problemi si introduce il concetto di **Soft Margin**.

Ora alcuni punti possono:

- entrare nel margine;
- persino trovarsi dal lato sbagliato dell'iperpiano.

L'algoritmo cerca un compromesso tra:

- margine molto ampio;
- pochi errori di classificazione.

Questo compromesso è controllato dal parametro **C**.

Il parametro C

Il parametro **C** controlla la regolarizzazione della SVM.

`C elevato`

- penalizza molto gli errori;
- margine stretto;
- pochi errori sul training set;
- maggiore rischio di overfitting.

`C basso`

- tollera alcune violazioni;
- margine più ampio;
- maggiore capacità di generalizzazione;
- migliore robustezza al rumore.

In pratica:

- **C alto → modello rigido**
- **C basso → modello più flessibile**

SVM lineari

Quando i dati sono linearmente separabili è sufficiente una **Linear SVM**.

Classificazione non lineare

Molti dataset reali non sono separabili con una retta o un iperpiano.

Una soluzione consiste nell'aggiungere nuove feature.

Ad esempio: $x_2 = x_1^2$

Così un dataset inizialmente non separabile può diventarlo in uno spazio di dimensione superiore.

Polynomial Features

Si possono creare automaticamente nuove feature. Il limite è che il numero delle feature cresce molto rapidamente all'aumentare del grado del polinomio.

Kernel Trick

Il **Kernel Trick** è uno dei concetti più importanti delle SVM.

Permette di lavorare come se i dati fossero stati trasformati in uno spazio ad altissima dimensionalità **senza calcolare esplicitamente le nuove feature**.

I vantaggi sono:

- minore memoria;
- minori tempi di calcolo;
- possibilità di utilizzare trasformazioni molto complesse.

È il motivo principale del successo delle SVM non lineari.

Polynomial Kernel

Il kernel polinomiale permette di simulare automaticamente le feature polinomiali.

Gli iperparametri principali sono:

- degree
- coef0
- C

Un grado troppo alto può provocare overfitting.

Un grado troppo basso può produrre underfitting.

Similarity Features

Un'altra strategia consiste nel descrivere ogni punto tramite la sua somiglianza con alcuni **landmark**.

La similarità è misurata mediante la funzione **Gaussian Radial Basis Function (RBF)**:

$\phi(x, l) = e^{-\gamma||x-l||^2}$

Ogni landmark genera una nuova feature.

Dopo questa trasformazione dati inizialmente non separabili possono diventare linearmente separabili.

Gaussian RBF Kernel

Anziché costruire esplicitamente tutte le similarity features, si utilizza il **kernel RBF**.
È probabilmente il kernel più usato.
Gli iperparametri principali sono:

- γ **(gamma)**: determina il raggio d'influenza di ogni punto;
- **C**: controlla la regolarizzazione.

Effetti di γ:

- γ alto → confini molto complessi → rischio overfitting;
- γ basso → confini più semplici → possibile underfitting.

Complessità computazionale

Le due implementazioni principali sono:

`LinearSVC`

- basata su **liblinear**;
- non usa kernel;
- complessità circa: $O(m \cdot n)$

dove:

- m = numero di esempi;
- n = numero di feature.

`SVC`

- basata su **libsvm**;
- supporta il Kernel Trick;
- complessità tra: $O\left(m^2 n\right)$ e $O\left(m^3 n\right)$

Per dataset molto grandi diventa quindi piuttosto lenta.

SVM per la regressione (SVR)

Le SVM possono essere utilizzate anche per problemi di regressione. In questo caso l'obiettivo non è separare classi, ma trovare una funzione che lasci il maggior numero possibile di punti all'interno di una fascia di ampiezza ε **(epsilon)**.
Solo i punti esterni a questa fascia contribuiscono all'errore. Questa proprietà è detta ε-insensitive.

Funzione decisionale

Per una classificazione lineare la decisione è presa calcolando: $f(x) = \omega^T x + b$
Se:

- $f(x) \geq 0 \rightarrow$ classe positiva;
- $f(x) < 0 \rightarrow$ classe negativa.

L'iperpiano decisionale è definito da: $\omega^T x + b = 0$
I margini corrispondono invece ai valori:
$\omega^T x + b = \pm 1$
L'addestramento consiste nel trovare ω e b che massimizzano il margine, consentendo eventualmente alcune violazioni nel caso della Soft Margin.

Obiettivo dell'addestramento

La SVM cerca di minimizzare la norma del vettore dei pesi: $\frac{1}{2}||\omega||^2$ poiché: pesi piccoli ⇒ margine grande.
Nella Soft Margin vengono introdotte le **slack variables** ζ_i, che misurano quanto ogni punto viola il margine.

La funzione obiettivo diventa: $\frac{1}{2}||\omega||^2 + C\sum_i \zeta_i$

Il parametro **C** bilancia quindi due obiettivi opposti:

- aumentare il margine;
- ridurre gli errori di classificazione.

Programmazione Quadratica e problema duale

L'addestramento di una SVM è formulato come un problema di **Quadratic Programming (QP)**, cioè un'ottimizzazione con funzione obiettivo quadratica e vincoli lineari. È possibile risolvere il problema sia nella forma **primal** sia nella forma **dual**. La formulazione duale è spesso più efficiente quando il numero di esempi è inferiore al numero di feature e, soprattutto, rende possibile l'applicazione del **Kernel Trick**, permettendo di gestire separazioni non lineari senza espandere esplicitamente lo spazio delle feature.

Online SVM

Le SVM possono essere addestrate anche in modalità incrementale (**online learning**), ad esempio con **SGDClassifier** per il caso lineare. In questa modalità si minimizza una funzione di costo che combina:

- una penalizzazione sulla norma dei pesi, per mantenere un margine ampio;
- una penalizzazione delle violazioni del margine (hinge loss).

Le versioni online convergono generalmente più lentamente dei metodi basati sulla programmazione quadratica, ma consentono di aggiornare il modello man mano che arrivano nuovi dati.

- Le SVM cercano l'**iperpiano con margine massimo**.
- I **Support Vectors** sono gli unici punti che determinano la soluzione.
- **Hard Margin**: nessun errore, richiede dati perfettamente separabili.
- **Soft Margin**: ammette alcune violazioni del margine.
- **C** controlla il compromesso tra ampiezza del margine ed errori.

- Le SVM richiedono **feature normalizzate**.
- Il **Kernel Trick** consente di risolvere problemi non lineari senza creare esplicitamente nuove feature.
- I kernel più utilizzati sono **Lineare**, **Polinomiale** e **RBF (Gaussiano)**.
- Per la regressione si utilizza **SVR**, che introduce il parametro **ε** per definire una fascia di tolleranza attorno alla funzione stimata.
- **LinearSVC** è più veloce e adatta a dataset molto grandi, mentre **SVC** con kernel è più flessibile ma ha un costo computazionale significativamente maggiore.

Cos'è un Decision Tree

Un Decision Tree rappresenta una sequenza di domande sui dati.

- Ogni **nodo interno** contiene una condizione su un attributo.
- Ogni **ramo** rappresenta l'esito della condizione.
- Ogni **foglia** contiene la predizione finale.

L'obiettivo è dividere progressivamente il dataset fino ad ottenere gruppi il più possibile omogenei.

Come effettuare una previsione

Per classificare una nuova istanza:

1. si parte dalla radice;
2. si verifica la condizione;
3. si sceglie il ramo corretto;
4. si continua fino ad arrivare ad una foglia.

La foglia contiene direttamente la classe predetta.
L'intero processo consiste quindi nell'attraversare il percorso dell'albero dalla radice fino ad una foglia.

Informazioni contenute nei nodi

Ogni nodo contiene alcune informazioni importanti.

`Samples`

Indica quante istanze del training raggiungono quel nodo.

`Value`

Mostra quante istanze appartengono ad ogni classe. La classe più numerosa sarà la predizione del nodo.

`Gini`

Misura quanto il nodo è "impuro". Formula:

$G = 1 - \sum p_i^2$ | p_i è la proporzione della classe i.

Interpretazione:

- **Gini = 0** → nodo perfettamente puro.
- **Gini alto** → classi molto mescolate.

L'obiettivo dell'algoritmo è ottenere nodi con impurità sempre minore.

Gini Split

Quando un nodo viene diviso, la qualità della divisione è valutata mediante il **Gini Split**.

Formula: $Gini_{split} = \frac{n_1}{n} G(T_1) + \frac{n_2}{n} G(T_2)$ dove:

- T_1 e T_2 sono i sottoinsiemi;
- n_1 e n_2 il numero di esempi.

Una divisione è tanto migliore quanto **più basso è il Gini Split**.

Confini decisionali

Ogni split genera una nuova frontiera di decisione.
Nel dataset Iris:

- il primo split è una linea verticale;
- il secondo una linea orizzontale;
- aumentando `max_depth` vengono aggiunti ulteriori confini.

Gli alberi costruiscono quindi regioni rettangolari nello spazio delle feature.

Stima delle probabilità

Un Decision Tree non restituisce solo la classe finale. Può anche stimare la probabilità di appartenenza ad ogni classe.
La probabilità è semplicemente il rapporto tra il numero di esempi della classe e il totale di esempi della foglia. La classe predetta è quella con probabilità maggiore.

Come si costruisce un Decision Tree

L'obiettivo è trovare un albero:

- piccolo;
- semplice;
- coerente con il training set.

L'algoritmo procede ricorsivamente:

1. sceglie il miglior attributo;
2. divide il dataset;
3. ripete il procedimento per ogni sottoinsieme;
4. termina quando:
    - tutti gli esempi appartengono alla stessa classe;
    - non rimangono attributi;
    - il sottoinsieme è vuoto.

Entropia

Un altro criterio per valutare uno split è l'**Entropia**, che misura il disordine del dataset.

Formula: $H = -\sum p_i log_2(p_i)$

Proprietà:

- Entropia = 0 → nodo puro.
- Entropia massima → classi distribuite uniformemente.

L'algoritmo sceglie l'attributo che riduce maggiormente l'entropia residua (massimizza l'Information Gain).

CART (Classification And Regression Trees)

Funzionamento:

1. prova tutti gli attributi;
2. prova tutte le possibili soglie;
3. calcola l'impurità dei due sottoinsiemi;
4. sceglie lo split con costo minimo.

Successivamente applica lo stesso procedimento ricorsivamente.
Il problema di trovare l'albero ottimale è **NP-Completo**, quindi CART utilizza un approccio greedy (ottimo locale).

Gini vs Entropia

Entrambi misurano l'impurità.

`Gini`

- più veloce;
- usato di default;
- leggermente più efficiente.

`Entropia`

- deriva dalla teoria dell'informazione;
- misura l'informazione contenuta nei dati;
- talvolta produce alberi leggermente più bilanciati.

Le prestazioni sono generalmente molto simili.

Tipi di split

Per variabili numeriche: $A \leq v$ e $A > v$
oppure più intervalli.
Per variabili categoriche:

- split binari;
- split multipli.

L'obiettivo è creare sottoinsiemi il più possibile omogenei.

Complessità computazionale

`Predizione`

Per classificare un nuovo esempio basta attraversare il percorso dell'albero.
Complessità: $O(log\ m)$ dove m è il numero di esempi. La predizione è quindi molto veloce.

`Addestramento`

Durante il training vengono valutati:

- tutte le feature;
- molti possibili split.

Complessità: $O(n \times m\ log\ m)$ dove:

- n = numero di feature;
- m = numero di esempi.

Overfitting e regolarizzazione

Gli alberi sono modelli **non parametrici** e, se lasciati crescere senza limiti, tendono ad adattarsi perfettamente al training set, causando **overfitting**.
Per limitarlo si usano diversi iperparametri:

- **max_depth** → profondità massima;
- **min_samples_split** → minimo numero di esempi per dividere un nodo;
- **min_samples_leaf** → minimo numero di esempi per foglia;
- **max_leaf_nodes** → massimo numero di foglie;
- **max_features** → massimo numero di feature considerate ad ogni split.

Ridurre la complessità dell'albero migliora spesso la capacità di generalizzazione.

Decision Tree per regressione

Gli alberi possono essere usati anche per problemi di regressione. La differenza principale è che:

- nella classificazione la foglia contiene una **classe**;
- nella regressione contiene un **valore numerico**.

La previsione è la **media dei valori target** degli esempi presenti nella foglia.

CART per regressione

Nel caso della regressione, CART non minimizza l'impurità ma il **Mean Squared Error (MSE)**.
Ogni split cerca quindi di creare regioni in cui i valori target siano il più possibile simili. Anche in regressione gli alberi possono andare facilmente in overfitting, per cui è consigliabile usare parametri di regolarizzazione come `min_samples_leaf`.

Limiti dei Decision Trees

I principali svantaggi sono:

- **Overfitting** se l'albero cresce troppo.
- **Decision boundaries ortogonali**, poco adatte a separazioni oblique.
- **Elevata instabilità**: piccole variazioni nei dati possono produrre alberi molto diversi.
- L'algoritmo CART è **stocastico**, quindi addestramenti differenti possono generare modelli differenti.

Per ridurre questa instabilità si utilizzano le **Random Forests**, che combinano le predizioni di molti alberi.

- I Decision Trees sono modelli supervisionati per classificazione e regressione.
- Ogni nodo esegue un test su una feature e ogni foglia produce una previsione.
- La qualità degli split è valutata principalmente con **Gini** o **Entropia**.
- Scikit-Learn implementa l'algoritmo **CART**, che costruisce l'albero in modo greedy.
- Le probabilità di classe sono calcolate dalla distribuzione delle classi nella foglia raggiunta.
- Le predizioni sono molto rapide (**O(log m)**), mentre l'addestramento è più costoso (**O(n·m·log m)**).
- Gli alberi sono altamente interpretabili ma soggetti a **overfitting** e **instabilità**, mitigabili con la regolarizzazione o con metodi ensemble come le Random Forests.

Gini ed Entropia misurano quanto un nodo è "sporco". Un nodo è puro quando contiene esempi di una sola classe, e in quel caso entrambi valgono 0. Durante la costruzione dell'albero vengono valutati diversi split e viene scelto quello che produce i sottoinsiemi più puri. Gini minimizza l'impurità, mentre con l'entropia si cerca di massimizzare l'Information Gain.

Random Forests & Ensemble Learning

Ensemble Learning

L'**Ensemble Learning** consiste nel combinare le predizioni di più modelli (*predictors*) per ottenere un modello complessivamente migliore.
L'idea è simile alla **"wisdom of the crowd"**: aggregando le risposte di molti modelli si può ottenere una previsione migliore di quella del singolo modello. Un insieme di modelli prende il nome di **ensemble**.
Se i modelli commettono errori diversi e poco correlati, gli errori dei singoli modelli possono compensarsi durante l'aggregazione.

Voting Classifiers

Supponiamo di avere diversi classificatori, ad esempio:

- Logistic Regression
- SVM
- Random Forest
- K-Nearest Neighbors

Possiamo combinare le loro predizioni.

`Hard Voting`

Ogni classificatore vota per una classe e viene scelta quella con **più voti**.
È il **majority vote**.
Un ensemble può essere più accurato anche se i singoli classificatori sono relativamente deboli.
Il vantaggio è maggiore quando i classificatori fanno errori differenti.
Se tutti sono addestrati sugli stessi dati e commettono gli stessi errori, il voto non migliora molto il risultato.

`Soft Voting`

Se i classificatori producono probabilità, si possono **mediare le probabilità** e scegliere la classe con probabilità media maggiore.
Il soft voting spesso funziona meglio dell'hard voting perché tiene conto anche della **confidenza** dei classificatori.

Bagging e Pasting

Invece di utilizzare algoritmi diversi, possiamo usare **lo stesso algoritmo più volte**, ma addestrando ogni modello su un sottoinsieme casuale diverso del training set.
La differenza fondamentale è il modo in cui vengono selezionati i dati:

Metodo	Campionamento
Bagging	con replacement
Pasting	senza replacement

`Bagging`

Un esempio può essere estratto più volte **anche per lo stesso modello**.

`Pasting`

Un esempio può comparire più volte nell'ensemble, ma **non più volte nello stesso sottoinsieme**.

Aggregazione

Una volta addestrati tutti i modelli:

- **classificazione** → si usa la moda / voto maggioritario
- **regressione** → si usa la media

L'aggregazione riduce soprattutto la **varianza**.
Il grande vantaggio è anche che i diversi modelli possono essere addestrati **in parallelo**, sfruttando più CPU o server.

Bagging vs Pasting

Il bagging introduce maggiore diversità tra i modelli. Questo può aumentare leggermente il bias, ma riduce la correlazione tra i modelli e quindi la varianza dell'ensemble.
Per questo motivo il **bagging è generalmente preferito**.

Out-of-Bag (OOB) Evaluation

Nel bagging, poiché si fa campionamento con replacement, non tutti gli esempi del training set vengono utilizzati da ogni albero.
In media:

- circa **63%** degli esempi viene utilizzato;
- circa **37%** rimane fuori.

Questi ultimi sono chiamati **Out-of-Bag (OOB) instances**.
Gli esempi OOB possono essere utilizzati per valutare il modello perché **quel particolare predictor non li ha mai visti durante l'addestramento**.
Quindi possiamo ottenere una stima delle prestazioni senza creare necessariamente un validation set separato o utilizzare cross-validation.

Random Patches e Random Subspaces

Il bagging può essere applicato non solo agli **esempi**, ma anche alle **feature**.
I parametri principali sono:

- `max_features`
- `bootstrap_features`

Ogni predictor viene quindi addestrato su un sottoinsieme casuale delle feature.

Random Patches

Si campionano casualmente: **istanze + feature**
È particolarmente utile con input ad alta dimensionalità, ad esempio immagini.

Random Subspaces

Si mantengono tutte le istanze ma si seleziona casualmente un sottoinsieme delle feature.

Random Forest

Una **Random Forest** è un ensemble di **Decision Trees**. Generalmente gli alberi vengono addestrati usando il **bagging**.
La caratteristica fondamentale è che introduce **ulteriore casualità nella costruzione degli alberi**.
Quando un albero deve effettuare uno split, non considera necessariamente tutte le feature, ma sceglie la migliore tra un **sottoinsieme casuale di feature**.
Questo aumenta la **diversità degli alberi**.

`Effetto`

Più diversità → alberi meno correlati → **minore varianza**.
Si accetta un aumento del bias per ottenere un modello complessivamente migliore.

Extra-Trees

Gli **Extra-Trees (Extremely Randomized Trees)** introducono ancora più casualità.
Nella Random Forest:

1. si seleziona casualmente un sottoinsieme di feature;
2. si cerca il **miglior threshold** per lo split.

Negli Extra-Trees:

1. si seleziona casualmente un sottoinsieme di feature;
2. vengono scelti anche **threshold casuali**, invece di cercare il migliore.

`Conseguenza`

Più casualità:
↑ bias → ↓ varianza
Inoltre gli Extra-Trees sono **più veloci da addestrare**, perché trovare il miglior threshold è una delle operazioni più costose nella costruzione di un albero.

Feature Importance

Uno dei vantaggi delle Random Forest è la possibilità di stimare l'**importanza delle feature**.
Il risultato viene normalizzato in modo che: **somma delle importanze = 1**.

Boosting

Il **Boosting** è un'altra famiglia di Ensemble Methods.
L'idea è combinare diversi **weak learners** per creare un **strong learner**.
A differenza del bagging, i predictor vengono generalmente addestrati **sequenzialmente**:
modello 1 → modello 2 → modello 3 → ...
Ogni nuovo modello cerca di correggere gli errori del precedente.
I due metodi principali presentati sono:
AdaBoost; Gradient Boosting

AdaBoost

AdaBoost cerca di concentrarsi progressivamente sugli esempi **più difficili**.
Funzionamento:

1. si addestra un primo classificatore;
2. si individuano gli esempi classificati male;
3. si aumenta il loro **peso**;
4. si addestra un nuovo classificatore;
5. si aggiornano nuovamente i pesi;
6. si ripete il processo.

Quindi gli esempi difficili diventano progressivamente più importanti.
Alla fine i predictor vengono combinati, ma hanno **pesi diversi** in funzione della loro accuratezza.

`Learning rate`

Il `learning_rate` controlla quanto vengono modificati i pesi a ogni iterazione.
Un learning rate più basso rende l'apprendimento più graduale.

`Gradient Boosting`

Anche il Gradient Boosting costruisce i modelli **sequenzialmente**.
La differenza principale rispetto ad AdaBoost è:

- **AdaBoost** → modifica i pesi degli esempi;
- **Gradient Boosting** → cerca di prevedere i **residui/errori** del modello precedente.

In altre parole, ogni nuovo predictor cerca di correggere gli errori rimasti.

`Learning Rate e Shrinkage`

Il `learning_rate` determina quanto ogni nuovo albero contribuisce al modello finale.
Un valore basso, ad esempio `0.1`:

- richiede più alberi;
- generalmente migliora la generalizzazione.

Questa tecnica di regolarizzazione viene chiamata **shrinkage**.
Troppi alberi possono comunque portare a **overfitting**.

Early Stopping

Per scegliere il numero ottimale di alberi si può usare l'**early stopping**.
L'idea è:

1. si addestra il modello con molti alberi;
2. si valuta l'errore di validazione dopo ogni fase;
3. si individua il numero di alberi che produce il **minimo validation error**;
4. si costruisce il modello finale utilizzando quel numero di alberi.

Stochastic Gradient Boosting

Il Gradient Boosting può anche utilizzare solo una **frazione casuale degli esempi** per addestrare ogni albero.
Effetti:

- ↑ **bias**
- ↓ **varianza**
- addestramento più veloce

Questa tecnica è chiamata **Stochastic Gradient Boosting**.

Stacking

Lo **Stacking (stacked generalization)** utilizza un altro modello per combinare le predizioni dei vari predictor.
Invece di fare semplicemente: "vince la classe con più voti", si addestra un **meta-learner**, detto anche **blender**, che impara come combinare le predizioni.

Esempio

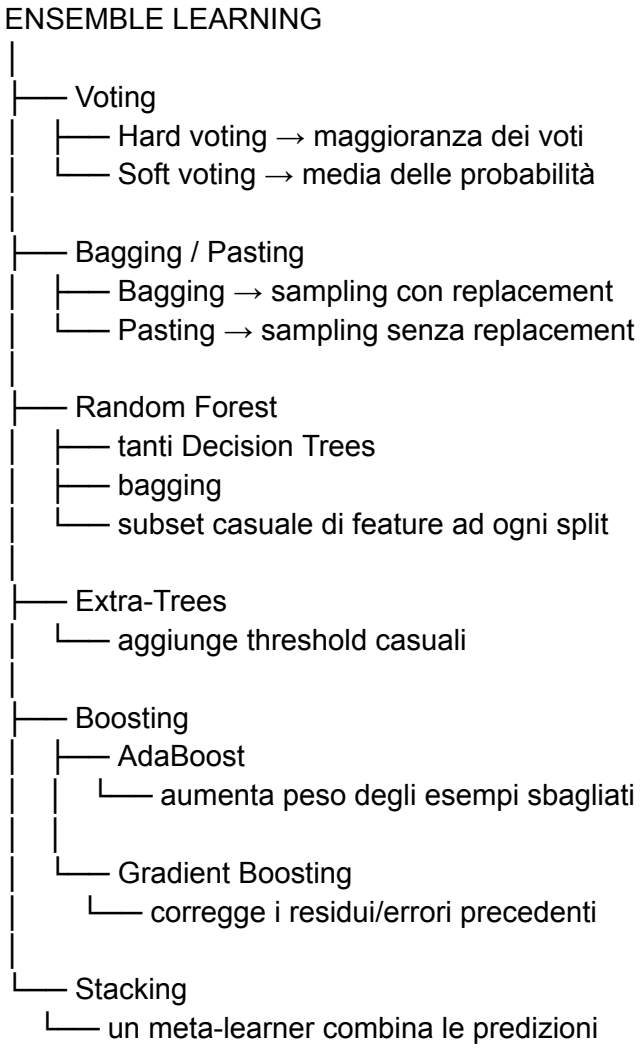
Tre modelli producono:

- modello 1 → 3.1
- modello 2 → 2.7
- modello 3 → 2.9

Il blender riceve: [3.1, 2.7, 2.9] e produce, ad esempio: 3.0

Come si addestra il blender?

Il training set viene diviso.
Una parte serve per addestrare i predictor del **primo livello**.
I predictor vengono poi utilizzati sull'**hold-out set**, che non hanno mai visto.
Le loro predizioni diventano le nuove feature di training per il blender.
Questo è importante perché le predizioni utilizzate per addestrare il blender devono essere **"clean"**, cioè prodotte su dati che i predictor non hanno utilizzato durante il proprio training.
È possibile anche creare più livelli di blender, formando uno **stacking multilayer**.



- **Ensemble Learning** = combinare più modelli per ottenere prestazioni migliori.
- **Bagging** = campionamento con replacement; invece di addestrare **un solo modello** su tutto il dataset, ne addestriamo **molti**, ciascuno su un campione casuale del dataset, e poi **combiniamo le loro predizioni**.
- **Pasting** = senza replacement.
- **Random Forest** = ensemble di Decision Trees + bagging + selezione casuale delle feature agli split.
- **Extra-Trees** = ancora più casualità, anche nei threshold.
- **Boosting** = modelli sequenziali che cercano di correggere gli errori precedenti.
- **Stacking** = le predizioni dei modelli diventano input di un **meta-learner**.

Dimensionality Reduction

Perché ridurre la dimensionalità?
Nei problemi di Machine Learning possiamo avere dataset con **migliaia o milioni di feature**. Un numero molto elevato di dimensioni:

- rende il training molto più lento;
- rende più difficile trovare una buona soluzione;
- aumenta il rischio di **overfitting**;
- può rendere le predizioni meno affidabili.

Questo problema è chiamato **Curse of Dimensionality** (maledizione della dimensionalità).
Ridurre la dimensionalità significa trasformare i dati da uno spazio con molte dimensioni a uno con meno dimensioni, cercando di perdere meno informazione possibile.
La riduzione dimensionale **perde inevitabilmente una parte dell'informazione**. Può velocizzare il training, ma può anche peggiorare leggermente le prestazioni e rendere la pipeline più complessa.

Curse of Dimensionality

In uno spazio ad alta dimensionalità il comportamento dei dati diventa molto diverso rispetto alla nostra intuizione tridimensionale.
Per esempio, in un ipercubo di dimensione molto elevata, la maggior parte dei punti si trova **vicino ai bordi**. Inoltre, aumentando le dimensioni, la distanza media tra i punti cresce enormemente.
Conseguenza importante: I dataset ad alta dimensionalità diventano molto **sparsi**.
I punti di training tendono a essere molto lontani tra loro e un nuovo punto può trovarsi lontano da tutti i punti disponibili. Le predizioni diventano quindi meno affidabili e aumenta il rischio di **overfitting**.
In teoria si potrebbe aumentare il numero di esempi di training per rendere il dataset più denso, ma il numero di esempi necessario cresce **esponenzialmente** con il numero di dimensioni.

Due approcci principali

Projection

Nei dataset reali, spesso le feature non sono tutte indipendenti:

- alcune sono quasi costanti;
- altre sono fortemente correlate.

Di conseguenza, anche se i dati sono rappresentati in uno spazio ad alta dimensionalità, possono trovarsi vicino a un **sottospazio di dimensionalità molto più bassa**.

`Problema della projection`

La proiezione non funziona sempre bene
Il dataset potrebbe avere una struttura curva, come nel famoso **Swiss roll**. Una semplice proiezione

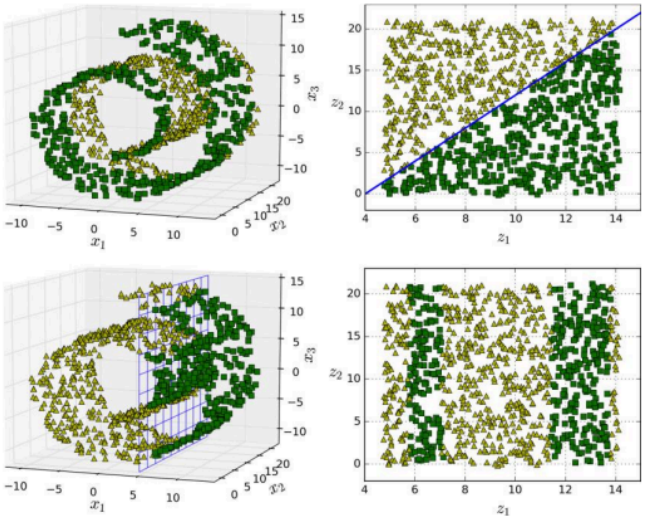
3D → 2D schiaccerebbe insieme parti diverse del dataset. L'obiettivo corretto sarebbe invece **srotolare** il Swiss roll, ottenendo la sua struttura 2D.

Manifold Learning

Il **Swiss roll** è un esempio di **2D manifold** immerso in uno spazio 3D.
Un manifold di dimensione d è una parte di uno spazio n-dimensionale, con d<n, che **localmente assomiglia a un iperpiano di dimensione d**.

`Manifold hypothesis`

Il **Manifold Learning** si basa sull'idea che: molti dataset reali ad alta dimensionalità si trovano vicino a un manifold di dimensionalità molto più bassa.
Questa è chiamata **manifold hypothesis**.



`Esempio: MNIST`

Le immagini delle cifre scritte a mano hanno molte caratteristiche comuni:

- linee connesse;
- sfondo bianco;
- cifre generalmente centrate;
- strutture simili.

Quindi, anche se ogni immagine ha moltissime pixel, non tutte le combinazioni dei pixel sono possibili per rappresentare una cifra. I dati risultano quindi confinati in uno spazio effettivamente molto più piccolo.

Il manifold semplifica sempre il problema?

No.
Un'altra ipotesi del Manifold Learning è che il problema di Machine Learning diventi più semplice nello spazio a dimensionalità ridotta.
Questo può succedere: nel caso del Swiss roll, una separazione complessa nello spazio 3D può diventare una semplice **retta** dopo aver srotolato il manifold.
Ma non è sempre così.
In alcuni casi una decision boundary molto semplice nello spazio originale può diventare complessa dopo la trasformazione sul manifold.
Quindi la riduzione dimensionale **non garantisce** un modello più semplice o migliori prestazioni.

PCA — Principal Component Analysis

La **PCA** è l'algoritmo di dimensionality reduction più famoso.
L'idea fondamentale è:

- trovare l'iperpiano che meglio rappresenta i dati;
- proiettare i dati su quell'iperpiano.

Obiettivo: preservare la varianza

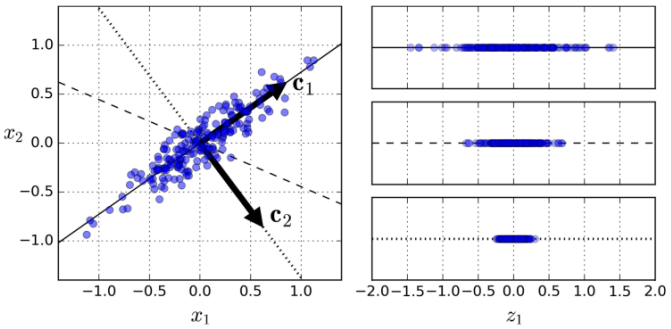
Tra tutti i possibili assi/iperpiani, PCA sceglie quello che **preserva la maggiore quantità di varianza**.
In altre parole, cerca la direzione lungo cui i dati sono maggiormente dispersi.
La prima direzione è quella che:

- massimizza la varianza conservata;
- equivalentemente, minimizza la distanza quadratica media tra dati originali e proiezioni.

Principal Components

PCA trova una sequenza di assi ortogonali:

- **PC1** → direzione con la maggiore varianza;
- **PC2** → direzione ortogonale a PC1 che conserva la maggiore varianza rimanente;
- **PC3** → direzione ortogonale alle precedenti con maggiore varianza rimanente;
- ecc.



Questi assi sono chiamati **Principal Components**.
Quindi: **PC1 > PC2 > PC3 > ...** in termini di quantità di varianza spiegata.
Una piccola perturbazione dei dati può cambiare il verso di una PC o, in alcuni casi, far ruotare/scambiare alcune componenti, ma generalmente il **sottospazio individuato rimane lo stesso**.

PCA e SVD

Per calcolare le Principal Components si può utilizzare la **Singular Value Decomposition (SVD)**.
La matrice dei dati X viene scomposta come:
$X = U\Sigma V^T$
La matrice V contiene le **Principal Components**.

Ridurre a *d* dimensioni

Una volta trovate tutte le Principal Components, possiamo scegliere solo le prime *d*.
Proiettiamo quindi i dati sull'iperpiano generato dalle prime *d* componenti.
Questo garantisce di conservare **la maggiore quantità possibile di varianza** per una rappresentazione di *d* dimensioni.
La formula della proiezione è:
$X_{d-proj} = XW_d$
dove W_d contiene le prime *d* Principal Components

Come scegliere il numero di dimensioni?

Una regola generale è scegliere il numero di componenti necessario a conservare una percentuale sufficientemente elevata della varianza, ad esempio **95%**.
Quindi non si sceglie necessariamente un numero arbitrario di componenti: si scelgono tante componenti quante ne servono per raggiungere la soglia desiderata.
Eccezione: se lo scopo è la **visualizzazione**, generalmente si scelgono 2 o 3 dimensioni.

Elbow della explained variance

Un altro metodo consiste nel rappresentare la **varianza cumulativa spiegata** in funzione del numero di dimensioni.
Solitamente compare un punto chiamato **elbow (gomito)**:

- prima dell'elbow la varianza cresce rapidamente;
- dopo l'elbow cresce molto più lentamente.

Questo punto può essere interpretato come una stima della **intrinsic dimensionality** del dataset.

La logica della lezione può essere riassunta così:
Molte feature
↓
Curse of Dimensionality
↓
Dataset sparso, training lento, maggiore rischio di overfitting
↓
Dimensionality Reduction
↓
Projection oppure **Manifold Learning**
↓
Tra gli algoritmi principali: **PCA**
↓
PCA cerca le direzioni di **massima varianza**
↓
PC1, PC2, PC3, ...
↓
Si scelgono le prime d componenti
↓
Si controlla la **explained variance ratio**
↓
Si sceglie d, spesso per conservare circa **95% della varianza**.

Curse of dimensionality → più dimensioni ⇒ dati più sparsi, training più difficile e rischio di overfitting.
Projection → proietta i dati su un sottospazio più piccolo.
Manifold Learning → sfrutta l'ipotesi che i dati ad alta dimensionalità siano vicini a un manifold di dimensione inferiore.
PCA → trova direzioni ortogonali che massimizzano progressivamente la varianza.
Numero di componenti → scegliere abbastanza PC da conservare una percentuale elevata della varianza, ad esempio **95%**.

Il neurone artificiale

McCulloch e Pitts proposero un modello molto semplice di **neurone artificiale**.
Inizialmente:

- gli input erano binari (`0/1`);
- l'output era binario;
- il neurone si attivava quando un numero sufficiente di input era attivo.

Successivamente nasce il **Perceptron**, introdotto da **Frank Rosenblatt nel 1957**. Gli input non sono più necessariamente binari. Il neurone artificiale viene chiamato **Linear Threshold Unit (LTU)**.

Come funziona un Perceptron

Il Perceptron calcola prima una **somma pesata degli input**: $z = w_1 x_1 + w_2 x_2 + \ldots + w_n x_n$ oppure $z = w^T x$ dove:

- x_i = valore dell'input;
- w_i = peso associato all'input;
- z = somma pesata.

Successivamente applica una **funzione di attivazione**, inizialmente una funzione a gradino:

$h_w(x) = step\left(w^T x\right)$

La funzione **Heaviside** restituisce tipicamente:

- `1` se l'input supera una certa soglia;
- `0` altrimenti.

`Esempio`

Con: $x = (0, 0, 1, 1)$
e: $w = (0.5, 0.3, 0.2, -0.4)$
si ottiene:
$z = 0(0.5) + 0(0.3) + 1(0.2) + 1(-0.4) = -0.2$
Applicando la funzione Heaviside: $f(-0.2) = 0$
Quindi l'output è **0**.

Funzioni di attivazione

`Step function`

Produce un output discreto, ad esempio 0 oppure 1. Problema: essendo piatta a tratti, **non fornisce un gradiente utile** per il Gradient Descent.

`Sigmoid / Logistic`

$\sigma(z) = \frac{1}{1+e^{-z}}$

Il risultato è compreso tra **0 e 1**, quindi è particolarmente utile quando l'output rappresenta una probabilità. È inoltre differenziabile.

`Tanh`

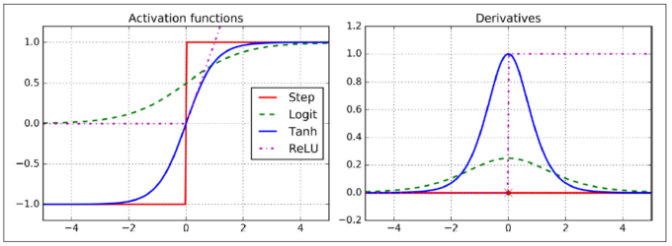
$tanh(z) = 2\sigma(2z) - 1$

L'output è compreso tra **−1 e 1**. Essendo centrata intorno a zero può favorire la convergenza durante il training.

`ReLU`

$ReLU(z) = max(0, z)$

È continua e molto veloce da calcolare. Non è differenziabile esattamente in z=0, ma **in pratica funziona molto bene** ed è ampiamente utilizzata nelle ANN.



Bias

Nelle reti può essere presente un **bias neuron**, che produce sempre 1.
Il bias permette alla rete di modificare la soglia/posizione della funzione di decisione e viene trattato come uno speciale neurone collegato agli altri.

Training del Perceptron

Il Perceptron viene addestrato attraverso una regola ispirata alla **Hebb's rule**.
L'idea è che una connessione tra neuroni che contribuisce frequentemente all'attivazione viene rafforzata.
La regola di aggiornamento dei pesi è:

$w_{i,j}^{(next)} = w_{i,j} + \eta\left(y_j - \hat{y}_j\right)x_i$

dove:

- $w_{i,j}$ = peso della connessione;
- η = **learning rate**;
- y_j = output corretto/target;
- $\hat{y}_j$ = output prodotto dalla rete;
- x_i = valore dell'input.

In pratica:

1. si presenta un esempio alla rete;
2. il Perceptron produce una previsione;
3. si confronta la previsione con il valore corretto;
4. se c'è un errore, si modificano i pesi per ridurlo.

Limite fondamentale del Perceptron

Il Perceptron ha una **decision boundary lineare**.
Quindi non è in grado di imparare pattern complessi che non siano **linearmente separabili**.
Un esempio classico è il problema **XOR**.
La cosa importante da ricordare è:
Perceptron → classificazione lineare.

Se i dati sono linearmente separabili, il **Perceptron Convergence Theorem** garantisce che l'algoritmo converge a una soluzione.

Multi-Layer Perceptron (MLP)

Per superare i limiti del Perceptron si possono mettere più neuroni/layer insieme.
Un **Multi-Layer Perceptron (MLP)** è costituito da:

1. **Input layer**
2. una o più **hidden layers**
3. **output layer**

I layer sono collegati e includono neuroni di bias.
La grande differenza è che un MLP può imparare relazioni **non lineari**, riuscendo per esempio a risolvere il problema XOR.
Quando una rete ha **almeno due hidden layers**, viene definita **Deep Neural Network (DNN)**.

Backpropagation

La domanda fondamentale diventa:
Come si addestra un MLP?
La soluzione è la **backpropagation**, introdotta nel 1986 da Rumelhart e altri autori. Oggi può essere interpretata come **Gradient Descent con reverse-mode autodiff**.
Il processo è:

`1. Forward pass`

L'input attraversa la rete e viene calcolato l'output di ogni neurone.

`2. Calcolo dell'errore`

Si confronta l'output prodotto con quello desiderato.

`3. Reverse pass`

L'errore viene propagato **all'indietro**, dalla parte finale verso l'input, per determinare quanto ogni connessione abbia contribuito all'errore.

`4. Aggiornamento dei pesi`

Si utilizza **Gradient Descent** per modificare i pesi e ridurre l'errore.
In forma molto semplice:
Input → Forward → Output → Errore → Backward → aggiornamento pesi

Classificazione

`Classificazione binaria`

L'output rappresenta una delle due classi.

`Classificazione multiclass`

Quando le classi sono mutuamente esclusive, viene utilizzata normalmente una **softmax** nell'output layer.
La **Softmax** prende un vettore di K valori e lo trasforma in una distribuzione di probabilità:

- ogni valore è tra 0 e 1;
- la somma delle probabilità è 1;
- si sceglie normalmente la classe con probabilità maggiore.

Cosa sono le CNN?

Le **Convolutional Neural Networks (CNN)** sono reti neurali progettate principalmente per lavorare con **immagini**, sia per problemi di **classificazione** sia di **regressione**. Il loro vantaggio è che richiedono relativamente poco preprocessing: sono infatti le CNN stesse ad **imparare i filtri** che negli algoritmi tradizionali dovevano essere progettati manualmente.
Le CNN si ispirano anche al funzionamento della corteccia visiva: gli esperimenti di **Hubel e Wiesel** mostrarono che molti neuroni rispondono a stimoli presenti solo in una piccola regione del campo visivo, detta **receptive field**. Regioni diverse possono sovrapporsi e coprire complessivamente tutto il campo visivo.

Perché non usare una normale rete neurale?

Una normale **Deep Neural Network (DNN)** può funzionare con immagini piccole, ma diventa rapidamente impraticabile con immagini grandi.
Ad esempio, un'immagine di **100×100 pixel** contiene 10.000 pixel; collegare tutti questi input a un livello successivo produce un numero enorme di connessioni e quindi di parametri.
Le CNN risolvono il problema sfruttando:

- **local connectivity** → ogni neurone guarda solo una piccola zona dell'immagine;
- **condivisione dei pesi** → lo stesso filtro viene utilizzato in diverse zone;
- progressiva estrazione di caratteristiche, da semplici a complesse.

Convolutional Layer

Il **convolutional layer** è il componente fondamentale di una CNN.
I neuroni del primo livello convoluzionale non sono collegati a tutti i pixel, ma soltanto ai pixel contenuti nel loro **receptive field**. In questo modo il primo livello può individuare caratteristiche semplici (**low-level features**), mentre i livelli successivi possono combinarle per ottenere caratteristiche più complesse (**higher-level features**).

`Receptive field`

Un neurone nella posizione `(i,j)` riceve informazioni da una piccola finestra della precedente feature map, di dimensione: $fh \times fw$ dove fh è l'altezza e fw la larghezza del receptive field.

`Padding`

Per mantenere le stesse dimensioni spaziali dell'input si possono aggiungere degli zeri attorno all'immagine: **Zero Padding**.

Stride

Lo **stride** indica di quanto viene spostato il filtro tra una posizione e la successiva.

- stride = 1 → il filtro si sposta di un pixel alla volta;
- stride = 2 → salta due pixel alla volta.

Uno stride maggiore permette quindi di ottenere un'uscita più piccola.

Pooling Layer

Il **pooling layer** serve principalmente a **ridurre le dimensioni** dell'immagine/feature map.
Questo permette di:

- diminuire il carico computazionale;
- ridurre l'utilizzo di memoria;
- diminuire il numero di parametri.

Un neurone di pooling considera una piccola regione dell'input e applica un'operazione di aggregazione.

Max Pooling

Nell'esempio delle slide:

- kernel: **2×2**
- stride: **2**
- nessun padding.

Viene mantenuto solamente il **valore massimo** della regione 2×2; gli altri valori vengono eliminati.

Struttura tipica di una CNN

Una CNN tipica segue uno schema simile:
Convolution → Pooling → Convolution → Pooling → ... → FNN → Output
Man mano che si attraversa la rete:

- le dimensioni spaziali dell'immagine diminuiscono;
- vengono estratte caratteristiche sempre più complesse;
- alla fine una normale rete **Feedforward Neural Network (FNN)** produce la predizione.

Principali architetture CNN

LeNet-5

LeNet-5 è una delle CNN più conosciute ed è stata utilizzata soprattutto per il riconoscimento di **cifre scritte a mano**, come nel dataset MNIST.
Caratteristiche:

- alternanza **Convolution → Pooling → Non Linearity**;
- **average pooling**;
- attivazione **sigmoid** negli strati intermedi;
- **tanh** nello strato F6;
- filtri convoluzionali **5×5**;
- 7 livelli;
- meno di 1 milione di parametri.

Lenta da addestrare, difficile da allenare e soffriva della scarsità di dati disponibile all'epoca.

AlexNet

AlexNet ha vinto l'ILSVRC del **2012** con un enorme miglioramento:

- AlexNet: **17% top-5 error**
- secondo classificato: **26%**.

Rispetto a LeNet-5 era:

- molto più grande;
- molto più profonda;
- innovativa perché introduceva la possibilità di mettere **più convolutional layers direttamente uno sopra l'altro**, senza inserire necessariamente un pooling dopo ogni convoluzione.

Regolarizzazione

Per limitare l'**overfitting**, AlexNet utilizzava:

- **Dropout al 50%** negli strati F8 e F9;
- **data augmentation**, modificando casualmente le immagini attraverso traslazioni, ribaltamenti orizzontali e variazioni dell'illuminazione.

Utilizzava inoltre una **Local Response Normalization** dopo ReLU in alcuni livelli.

GoogLeNet e Inception

GoogLeNet ha vinto l'ILSVRC **2014**, portando il top-5 error rate sotto il **7%**.
L'idea centrale sono gli **Inception modules**.
Invece di utilizzare una sola convoluzione, l'input viene copiato e inviato a diversi rami che utilizzano kernel differenti:

- **1×1**
- **3×3**
- **5×5**

I risultati vengono poi concatenati.
Un'importante innovazione è la **1×1 convolution**, utilizzata per ridurre la dimensionalità e quindi il costo computazionale.
GoogLeNet aveva circa **10 volte meno parametri di AlexNet**.
Gli output dei diversi rami possono essere concatenati perché vengono utilizzati stride 1 e **SAME padding**, mantenendo uguali altezza e larghezza.
La rete utilizza inoltre:

- ReLU;
- max pooling;
- average pooling;
- dropout;
- softmax finale per ottenere le probabilità delle classi.

Auxiliary classifiers

GoogLeNet introduceva anche due **classificatori ausiliari** all'interno della rete. Servivano a evitare che la parte centrale della rete "morisse", fornendo ulteriori segnali di training attraverso una loss ausiliaria.

ResNet

Con l'aumentare della profondità delle reti compare un problema importante:

Vanishing Gradient

Durante il **backpropagation**, il gradiente può diventare molto piccolo e molti neuroni possono smettere di apprendere ("die").

L'idea di **ResNet (Residual Network)** è introdurre delle **shortcut/skip connections**.

Invece di obbligare il segnale a passare attraverso tutti gli strati, una parte può "saltare" alcuni livelli.

La caratteristica fondamentale introdotta nella lezione è il salto di **2 layer**. Grazie alle skip connections, il segnale può propagarsi più facilmente attraverso tutta la rete.

Una ResNet può quindi essere vista come una sequenza di **residual units**, ciascuna contenente una piccola rete con una **skip connection**.

- **CNN → progettate principalmente per immagini.**
- **Receptive field → ogni neurone guarda solo una piccola zona.**
- **Convolution → estrae feature locali.**
- **Padding → permette di controllare/mantenere le dimensioni.**
- **Stride → determina quanto si sposta il filtro.**
- **Pooling → riduce dimensioni, memoria e computazione.**
- **Max pooling → mantiene il valore massimo della regione.**
- **CNN tipica → Convolution + Pooling ripetuti → FNN → output.**
- **LeNet-5 → MNIST, architettura storica.**
- **AlexNet → 2012, enorme miglioramento su ImageNet, dropout + data augmentation.**
- **GoogLeNet → 2014, Inception, convoluzioni 1×1/3×3/5×5, molti meno parametri.**
- **ResNet → skip connections per facilitare l'addestramento di reti molto profonde.**

LeNet → CNN classica → AlexNet → rete più grande/profonda → GoogLeNet → Inception e efficienza → ResNet → profondità + skip connections.

Recurrent Neural Networks

Cosa sono le RNN

Le **Recurrent Neural Networks (RNN)** sono reti neurali progettate per lavorare con **sequenze di dati**, anche di lunghezza arbitraria.

A differenza delle reti feedforward, una RNN possiede delle **connessioni ricorrenti**: l'output prodotto a un certo istante viene utilizzato come informazione per il passo temporale successivo.

Sono quindi adatte a problemi in cui **l'ordine e la storia degli input sono importanti**, ad esempio:

- serie temporali e prezzi azionari;
- traiettorie di veicoli autonomi;
- testo e linguaggio naturale;
- traduzione automatica;
- speech-to-text;
- sentiment analysis;
- generazione di testo, musica e didascalie.

Un problema importante delle RNN è quello dei **gradienti vanishing/exploding**, che viene affrontato soprattutto mediante architetture più sofisticate come **LSTM e GRU**.

Il neurone ricorrente

Il neurone ricorrente più semplice riceve, al tempo `t`:

- l'input corrente `x(t)`;
- l'output precedente `y(t-1)`.

Produce quindi `y(t)`, che viene nuovamente utilizzato al passo successivo.

La formula fondamentale è:

$$y^{(t)} = \phi\left(W_x^T x^{(t)} + W_x^T y^{(t-1)} + b\right)$$

dove:

- `x(t)` = input al tempo `t`;
- `y(t-1)` = output precedente;
- `Wx` = pesi associati all'input;
- `Wy` = pesi associati all'informazione precedente;
- `b` = bias;
- `φ` = funzione di attivazione, ad esempio `tanh`.

Idea fondamentale

Il valore `y(t)` dipende da `y(t-1)`, che dipende da `y(t-2)`, ecc.

Quindi: $y^{(t)} = f\left(x^{(t)}, x^{(t-1)}, \ldots, x^{(0)}\right)$

In altre parole, **la RNN mantiene una sorta di memoria degli input passati**.

Unrolling through time

Per capire una RNN si può "srotolare" (**unroll**) la rete lungo l'asse temporale.
Una singola cella viene quindi rappresentata come una sequenza di celle:

$x_0 \rightarrow [RNN] \rightarrow y_0$
$\downarrow$
$x_1 \rightarrow [RNN] \rightarrow y_1$
$\downarrow$
$x_2 \rightarrow [RNN] \rightarrow y_2$
$\downarrow$
...

In realtà le celle ai vari istanti **condividono gli stessi pesi** `Wx`, `Wy` e bias.

Al primo istante (`t=0`) non esiste uno stato precedente, quindi normalmente si assume:
$y^{(-1)} = 0$

Memory Cell e Hidden State

La capacità di una RNN di conservare informazioni nel tempo viene descritta attraverso una **memory cell**.

Lo **stato nascosto (hidden state)** al tempo `t` viene indicato con: $h^{(t)} = f\left(h^{(t-1)}, x^{(t)}\right)$
Quindi lo stato attuale dipende da:
stato precedente + input corrente.
È importante distinguere:

- **hidden state `h(t)`** → rappresenta lo stato interno della cella;
- **output `y(t)`** → ciò che la cella produce verso l'esterno.

Nelle celle RNN semplici possono coincidere, mentre nelle celle più complesse possono essere differenti.

Tipologie di input/output

Le RNN possono essere organizzate in diversi modi.

`Sequence → Sequence`

Esempio: $x_0\ x_1\ x_2\ x_3 \rightarrow$ RNN $\rightarrow y_0\ y_1\ y_2\ y_3$
È utile, ad esempio, per la previsione di **serie temporali**.

`Sequence → Vector`

Si fornisce una sequenza ma si utilizza soltanto l'ultimo output.
Esempio: "Questo film è davvero fantastico"

$\downarrow$
RNN
$\downarrow$
sentiment score

È adatto, ad esempio, alla **sentiment analysis**.

`Vector → Sequence`

Si fornisce un singolo input e si genera una sequenza.
Esempio: immagine → RNN → "un cane corre nel parco"
È il caso della **generazione di caption per immagini**.

`Encoder → Decoder`

È composta da due RNN:

1. **Encoder**: trasforma una sequenza in un vettore;
2. **Decoder**: trasforma il vettore in una nuova sequenza.

Esempio classico:
Frase inglese
$\downarrow$
Encoder
$\downarrow$
vettore
$\downarrow$
Decoder
$\downarrow$
Frase italiana
È il principio utilizzato per la **traduzione automatica**.

Static vs Dynamic Unrolling

`Static unrolling`

Con `static_rnn()` viene costruito esplicitamente un grafo con una cella per ogni timestep.
Le celle condividono i pesi.
Il problema è che con molti timestep il grafo diventa molto grande e può richiedere molta memoria durante la backpropagation.

`Dynamic unrolling`

`dynamic_rnn()` risolve il problema utilizzando un `while_loop()` per eseguire la cella il numero appropriato di volte.
Inoltre `swap_memory=True` permette di spostare parte della memoria dalla GPU alla CPU durante la backpropagation, riducendo il rischio di **Out Of Memory (OOM)**.

Sequenze di lunghezza variabile

Nella realtà le sequenze non hanno necessariamente tutte la stessa lunghezza.
Esempio:
"ciao" → 1 timestep
"ciao come stai" → 3 timestep
"ciao come stai oggi" → 4 timestep
La soluzione mostrata nella lezione è:

1. effettuare **padding** delle sequenze più corte;
2. fornire a `dynamic_rnn()` la lunghezza reale di ogni sequenza tramite `sequence_length`.

Esempio:
sequenza 1 → [a, b]
sequenza 2 → [c, 0] ← padding
sequenza 3 → [d, e]
con:
seq_length = [2, 1, 2]
La RNN ignora le parti oltre la lunghezza reale e produce vettori nulli per quei timestep.

Output di lunghezza variabile

Il problema è ancora più interessante quando **non conosciamo in anticipo la lunghezza dell'output**.
È tipico nella traduzione:
Input: "How are you?"
Output: "Come stai?"
Le due frasi possono avere lunghezze diverse.
La soluzione presentata è utilizzare un particolare simbolo:
`EOS — End Of Sequence`
Quando il decoder produce il token `EOS`, significa: "La sequenza è terminata."
Tutto ciò che viene prodotto dopo `EOS` viene ignorato.

Training delle RNN: BPTT

Per addestrare una RNN si utilizza la:
`Backpropagation Through Time (BPTT)`
L'idea è semplice:

1. **srotolare la RNN nel tempo**;
2. eseguire il **forward pass**;
3. calcolare la funzione di costo;
4. propagare i gradienti all'indietro attraverso tutti i timestep;
5. aggiornare i parametri.

Schema:
Forward:
$x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow x_4$

Backward:
$\leftarrow \quad \leftarrow \quad \leftarrow \quad \leftarrow$
gradienti
Poiché gli stessi pesi `W` e bias `b` sono utilizzati a ogni timestep, durante BPTT i contributi dei gradienti vengono **sommati attraverso i diversi timestep**.

Problema Vanishing/Exploding Gradient

Questo è uno dei problemi principali delle RNN.
Durante la propagazione del gradiente attraverso molti timestep, il gradiente può:

- diventare **molto piccolo** → **vanishing gradient**;
- diventare **molto grande** → **exploding gradient**.

Il vanishing gradient è particolarmente problematico perché rende difficile per la rete imparare **dipendenze a lungo termine**.
La lezione introduce quindi LSTM e GRU come soluzioni a questo problema.

RNN per le Time Series

Le RNN possono essere utilizzate per prevedere il valore successivo di una serie temporale.
Esempi:

- prezzo di un'azione;
- temperatura;
- segnali cerebrali;
- altre sequenze temporali.

L'idea del training è:
Input:
$x_0\ x_1\ x_2\ \ldots\ x_{19}$

Target:
$x_1\ x_2\ x_3\ \ldots\ x_{20}$
Quindi il target è semplicemente la sequenza di input **spostata di un timestep nel futuro**.

Funzione di costo

Per la previsione della serie temporale viene utilizzato il: **Mean Squared Error — MSE**

$$MSE = \frac{1}{n}\sum_i \left(y_i - \widehat{y}_i\right)^2$$

La lezione utilizza:

- **MSE** come loss;
- **Adam** come optimizer;
- learning rate `0.001`.

Durante l'addestramento l'MSE dell'esempio diminuisce progressivamente:
Iterazione 0 → MSE ≈ 13.65
Iterazione 100 → MSE ≈ 0.54
Iterazione 200 → MSE ≈ 0.17
Iterazione 300 → MSE ≈ 0.088
Iterazione 400 → MSE ≈ 0.063
Quindi il modello impara progressivamente a prevedere la serie temporale.

- **RNN**

Una rete neurale che lavora su **sequenze** e mantiene informazione degli input precedenti.

- **Formula fondamentale**

$$y_t = \phi\left(W_x^T x^{(t)} + W_x^T y^{(t-1)} + b\right)$$

- **Hidden state**

$h^{(t)} = f\left(h^{(t-1)}, x^{(t)}\right)$ È la **memoria** della rete.

- **Unrolling**

La RNN viene rappresentata come una sequenza di celle nel tempo, con **pesi condivisi**.

- **Sequenze variabili**

Si usa **padding +** `sequence_length`.

- **Output variabile**

Si usa un **EOS token**.

- **Training**

Si usa **Backpropagation Through Time (BPTT)**.

- **Problema principale**

Vanishing/exploding gradients → motivazione per LSTM/GRU.

- **Time series**

Input e target sono sequenze sfalsate di un timestep:

$X = \left(x_0, x_1, \ldots, x_{19}\right) \quad Y = \left(y_1, y_2, \ldots, y_{20}\right)$

Una RNN è una rete neurale per dati sequenziali che, a ogni timestep, combina l'input corrente con lo stato precedente per mantenere una memoria della sequenza; viene addestrata tramite Backpropagation Through Time, ma può soffrire di vanishing/exploding gradients.

Representation Learning

Le reti neurali non imparano soltanto a produrre un risultato finale, ma imparano anche delle **rappresentazioni interne dei dati**, dette *features* o *latent representations*. Gli **autoencoder** sfruttano proprio questa idea per imparare rappresentazioni dei dati **senza supervisione**, cioè senza bisogno di dati etichettati.
Gli autoencoder possono essere utilizzati per:

- **riduzione della dimensionalità** e compressione;
- estrazione di **features**;
- classificazione nello spazio latente;
- pre-training non supervisionato;
- generazione di nuovi dati.

Autoencoder

Un **autoencoder** è una rete neurale che cerca di **ricostruire il proprio input**.
È composto da due parti:
Input → Encoder → Latent representation → Decoder → Output

- **Encoder**: trasforma l'input in una rappresentazione compatta, chiamata *coding* o *latent representation*.
- **Decoder**: prende la rappresentazione latente e cerca di ricostruire l'input originale.
- L'output viene quindi chiamato **reconstruction**.

`Funzione di costo`

Il modello viene addestrato minimizzando una **reconstruction loss**, che misura quanto l'output ricostruito è diverso dall'input. Generalmente viene utilizzata una distanza basata su una norma.

`Undercomplete Autoencoder`

Se la rappresentazione latente ha **dimensione inferiore all'input**, si parla di **undercomplete autoencoder**.
Il collo di bottiglia (*bottleneck*) costringe la rete a conservare le informazioni più importanti e a eliminare quelle meno significative.
Meno dimensioni nello spazio latente → La rete deve imparare quali caratteristiche dell'input sono davvero importanti.

Autoencoder e PCA

Un risultato importante:
Se un autoencoder:

- utilizza **solo trasformazioni lineari**;
- viene addestrato con **Mean Squared Error (MSE)**;
- è undercomplete;

allora realizza sostanzialmente una **PCA (Principal Component Analysis)**.

Quindi un autoencoder può essere visto come una forma di **self-supervised learning**.

Stacked Autoencoders

Uno **stacked autoencoder**, o **deep autoencoder**, contiene più hidden layer.
L'utilizzo di più livelli permette di imparare rappresentazioni più complesse. Tuttavia, la rete non deve essere resa eccessivamente potente: se il modello è troppo capace, potrebbe imparare semplicemente a ricostruire perfettamente l'input senza imparare una rappresentazione utile.

`Visualizzazione`

Gli stacked autoencoder non sono necessariamente il metodo migliore per visualizzare dati in 2D.
Una strategia proposta nella lezione è:
Input → Autoencoder → spazio di dimensione 30 → t-SNE → 2D
In questo modo l'autoencoder gestisce la prima riduzione della dimensionalità e t-SNE quella finale per la visualizzazione.

Unsupervised Pretraining

Gli autoencoder possono essere utili quando abbiamo:

- **molti dati non etichettati**
- **pochi dati etichettati.**

Si può:

1. addestrare uno stacked autoencoder utilizzando tutti i dati, anche quelli senza label;
2. riutilizzare i layer inferiori dell'encoder;
3. costruire la rete destinata al task finale;
4. addestrarla utilizzando i dati etichettati.

I layer pre-addestrati possono anche essere **congelati** (*frozen*), almeno quelli inferiori.

Convolutional Autoencoders

Per le **immagini**, le CNN sono più adatte delle reti dense.
Un **convolutional autoencoder** utilizza:

`Encoder`

- convolution;
- pooling;
- riduzione di **altezza e larghezza**;
- aumento del numero di **feature maps**.

`Decoder`

Fa il processo inverso, utilizzando principalmente **transposed convolutions**.
Per il *downsampling* si possono utilizzare:

- strided convolutions;
- max pooling.

Per l'*upsampling*:

- interpolazione lineare/bilineare/bicubica;
- **transposed convolution**, indicata nella lezione come la soluzione più comune.

Recurrent Autoencoders

Per dati sequenziali si possono utilizzare **recurrent autoencoders**.
L'architettura è:
Sequenza → RNN encoder → vettore → RNN decoder → sequenza
L'encoder è tipicamente una RNN **sequence-to-vector**, mentre il decoder è una RNN **vector-to-sequence**.
Un `RepeatVector` permette di fornire il vettore latente al decoder a ogni time step.

Overcomplete Autoencoders

Finora si è usato un bottleneck più piccolo dell'input per costringere il modello a imparare caratteristiche significative.
Se invece il coding ha dimensione **uguale o superiore** a quella dell'input, abbiamo un **overcomplete autoencoder**.
Il problema è che, senza ulteriori vincoli, la rete potrebbe semplicemente imparare una copia dell'input.
Per questo vengono introdotti altri tipi di autoencoder.

Denoising Autoencoders

Un **denoising autoencoder** riceve in ingresso un dato **corrotto**, ma deve ricostruire il dato originale.
Schema:
Input pulito → aggiunta rumore → input rumoroso → Autoencoder → input pulito
Il rumore può essere, ad esempio:

- rumore gaussiano;
- input casualmente spenti, analogamente al dropout.

`Obiettivo`

Rendere encoder e decoder **robusti a piccole perturbazioni** dei dati.

Sparse Autoencoders

L'obiettivo di uno **sparse autoencoder** è ottenere una rappresentazione latente **sparsa**, cioè con pochi neuroni attivi.
L'idea è ottenere un vettore che descriva bene il dato utilizzando il minor numero possibile di componenti.
Questo può essere utile anche per la classificazione.
Per ottenere la sparsità si aggiunge alla funzione di costo un **termine di regolarizzazione**.

Un esempio è:

- sigmoid nel coding layer;
- regolarizzazione **L1** sulle attivazioni.

Un'altra strategia consiste nel fissare una **sparsità target** e penalizzare le attivazioni che si discostano da essa. Nella lezione viene utilizzata la **KL divergence** come sparsity loss.
La funzione di costo quindi comprende:
reconstruction loss + sparsity loss × sparsity weight

Variational Autoencoders (VAE)

I **Variational Autoencoders** sono particolarmente importanti perché sono **probabilistici e generativi**.
A differenza di un autoencoder classico, l'encoder non produce direttamente un singolo vettore latente.
Produce invece:

- **μ (media)**
- **σ (deviazione standard)**

Il vettore latente viene poi **campionato casualmente** da una distribuzione gaussiana:
$z \sim N(\mu, \sigma)$ e viene passato al decoder.

`Funzione di costo del VAE`

È composta da due componenti:

1. **Reconstruction loss** → ricostruire bene l'input.
2. **Latent loss** → rendere lo spazio latente simile a una distribuzione Gaussiana.

La seconda componente utilizza la **KL divergence**.

`Generazione`

Una volta addestrato il VAE, possiamo:

1. campionare casualmente un punto nello spazio latente;
2. passarlo al decoder;
3. ottenere un nuovo dato.

Per esempio, per Fashion MNIST si possono generare nuove immagini campionando codifiche casuali da una distribuzione gaussiana.

`Semantic interpolation`

Una caratteristica interessante è la **semantic interpolation**.
Invece di interpolare direttamente tra due immagini a livello di pixel:
immagine A → encoder → z_1
immagine B → encoder → z_2
poi si interpola tra **z_1 e z_2** e si passa il risultato al decoder.
In questo modo si ottiene una transizione semantica tra le due immagini.

GAN — Generative Adversarial Networks

Una GAN è costituita da **due reti neurali**:

`Generator`

Genera nuovi dati a partire da un vettore di rumore casuale.

`Discriminator`

Deve stabilire se un dato è:

- **reale** → proveniente dal dataset;
- **fake** → prodotto dal Generator.

Le due reti vengono addestrate contemporaneamente e **competono** tra loro.

Come funziona una GAN

Il processo è:
Random noise → Generator → immagine fake
Poi:
immagine fake + immagini reali → Discriminator
Il discriminator restituisce una probabilità compresa tra **0 e 1**:

- vicino a **1** → immagine considerata autentica;
- vicino a **0** → immagine considerata falsa.

Il Generator cerca quindi di produrre immagini sempre più realistiche per **ingannare il Discriminator**.
Il Discriminator, invece, cerca di diventare sempre più bravo a distinguere reale e falso.

Il Minmax Game

L'addestramento della GAN può essere visto come un **gioco min-max**:

- il **Discriminator** vuole massimizzare la capacità di distinguere reali e falsi;
- il **Generator** vuole minimizzare questa capacità, facendo sembrare i dati generati reali.

In particolare:

`Discriminator:`

- D(x) → vicino a **1** per dati reali;
- D(G(z)) → vicino a **0** per dati generati.

`Generator:`

- vuole che D(G(z)) → **1**.

Quindi le due reti hanno **obiettivi opposti**.

Autoencoder	Ricostruisce l'input attraverso uno spazio latente
Undercomplete AE	Latent space più piccolo → impara caratteristiche importanti
Stacked AE	Più hidden layer → rappresentazioni più complesse
Convolutional AE	Autoencoder adatto alle immagini
Recurrent AE	Autoencoder per sequenze
Denoising AE	Input rumoroso → ricostruzione dell'input pulito
Sparse AE	Pochi neuroni latenti attivi
VAE	Latent space probabilistico → permette generazione
GAN	Generator e Discriminator competono
Generator	Produce dati falsi cercando di renderli realistici
Discriminator	Distingue dati reali da dati generati